



数据库并行处理技术

张超

中国人民大学信息学院



目录

- 并发事务处理技术
- 并发查询处理技术
- 混合事务与查询处理技术



目录

- 并发事务处理技术
- 并发查询处理技术
- 混合事务与查询处理技术



1、并发事务处理技术

- **事务的定义：事务是多个数据库操作组合成的一个不可分割的、同时成功或失败的工作单元。**
 - 如下为一转账事务

Begin;

UPDATE account SET balance = balance + 100.00 WHERE name = '小王';

UPDATE account SET balance = balance - 100.00 WHERE name = '小明';

COMMIT;

- Begin表示事务开始
- COMMIT 表示事务提交
- ROLLBACK 表示事务回滚，撤销该事务



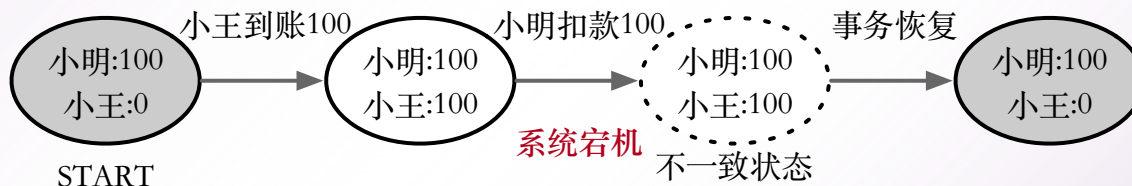
1、并发事务处理技术

- **数据库事务包含ACID四大特性。**

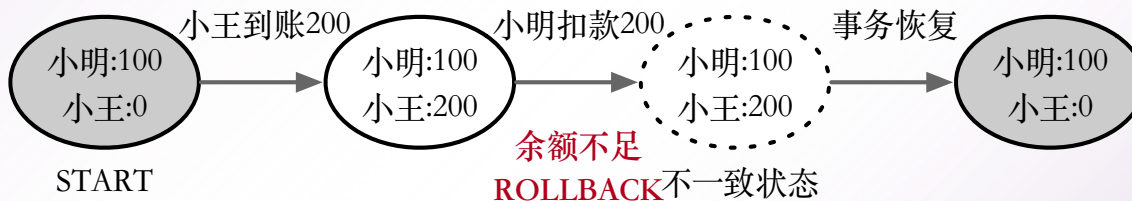
- 原子性 (atomicity) : **不可分割**, 事务单元全部成功或者全部失败
- 一致性 (consistency) : **正确一致**, 一个正确 (一致) 的状态转移到另一个正确的状态
- 隔离性 (isolation) : **互不干扰**, 多个事务在并发执行的过程中所得到的结果, 和串行执行得到的结果是一致
- 持久性 (durability) : **永久保持**, 执行结果不会丢失

1、并发事务处理技术

□ 给小明扣款100元时发生系统故障，该操作不成功，需要恢复所有的数据。



□ 给小明扣款200元时余额不足，主动回滚 (ROLLBACK)，需要恢复小王到账的200元



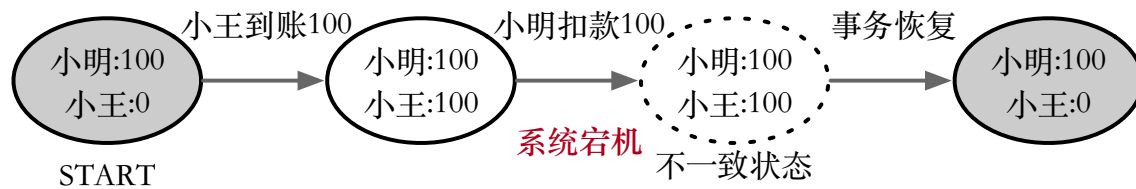
原子性



1、并发事务处理技术

□ 事务的执行会改变系统状态，一致性是指事务的执行会保证数据库从一个正确（一致）的状态转移到另一个正确的状态。

- 一致状态：比如数据库可以存储负数，但银行业务中用户的存款不能出现负数，存在负存款账户的数据库状态是不正确的状态。
- 下图中可以设置检查条件“小明和小王的存款和为100元”来对转账事务进行一致性的验证，宕机后的状态不满足一致性。



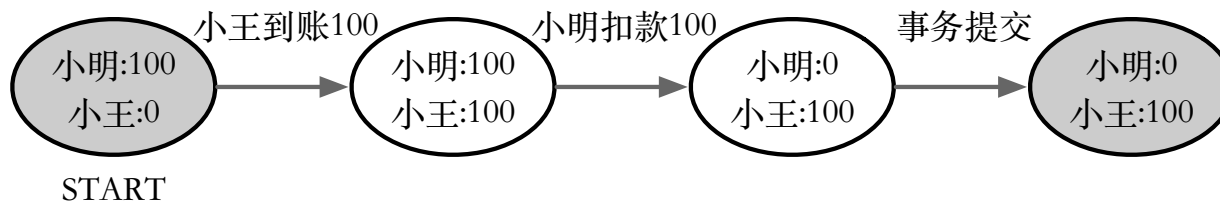
一致性



1、并发事务处理技术

□ 持久性是指，一旦事务完成提交，即使数据库发生故障，该事务的执行结果也不会丢失，可以被正确地恢复，仍然对后续事务可见。

- 下图描述的转账事务中，一旦转账事务提交，小明的100元就会永远地转给小王
- 即使后续系统发生故障，该100元也不会丢失，也不会转移回小明的账户上。



持久性



1、并发事务处理技术

- 事务的隔离性是指多个事务在并发执行的过程中所得到的结果，和串行执行得到的结果是一致的，保证了事务并发处理中事务的正确性
- 并发控制是实现事务隔离性的手段
 - 串行处理不存在事务之间的相互干扰，但是不能充分利用计算资源，效率低下。
 - 并行处理可以提升事务并发性，但是要解决冲突问题

隔离性



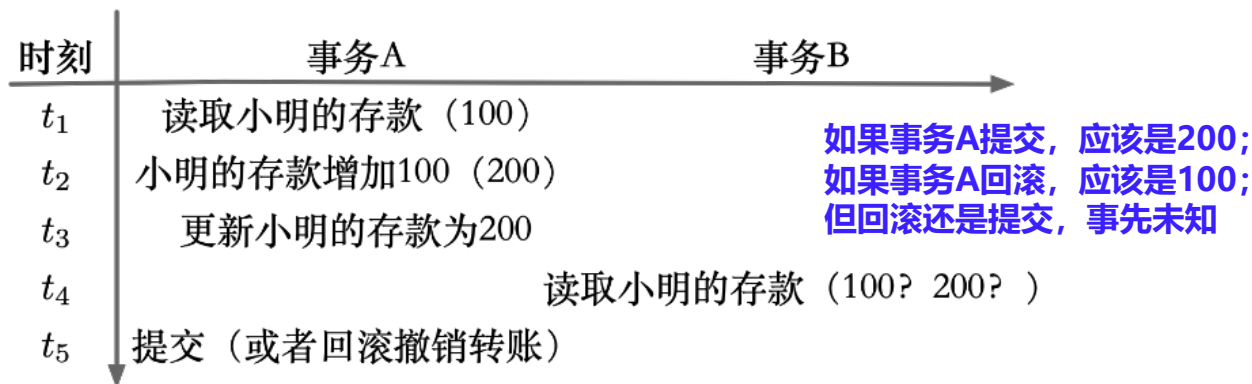
1、并发事务处理技术

□ 银行场景中新来了两个事务，事务A先被请求：

- 事务A，小明给账户存款100元。
- 事务B，家长查询小明存款。

□ 事务B与事务A在并发过程中存在写读冲突：导致脏读（dirty read）

- 事务A更新了小明的余额数据，使其从100元变成200元，但是事务A还未提交。
- 此时事务B要读取小明的余额数据，数据库应该读取原先的数据（100），还是事务A修改但是未提交的数据（200）？



隔离性-写读冲突



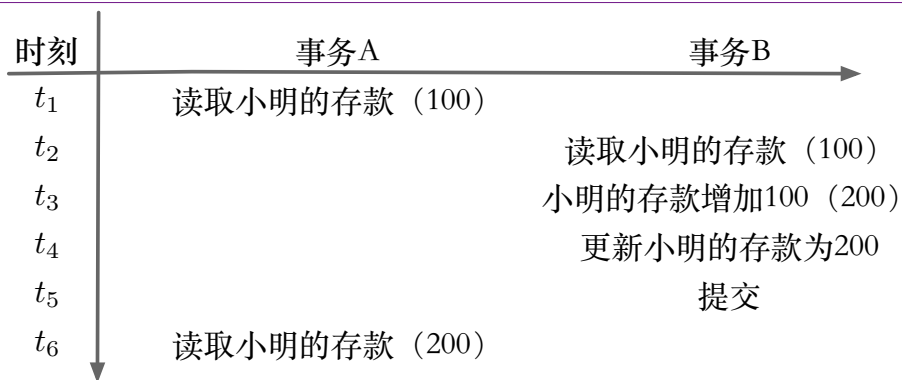
1、并发事务处理技术

□ 银行场景中新来了两个事务，事务A先被请求：

- 事务A，小明两次查询账户存款。
- 事务B，小明家长给小明存款100元。

□ 事务A与事务B在并发过程中存在读写冲突：导致不可重复度 (unrepeatable read)

- 事务A两次读取数据不一致。
- 事务B中间修改了数据造成其他事务的不可重复读。



隔离性-读写冲突



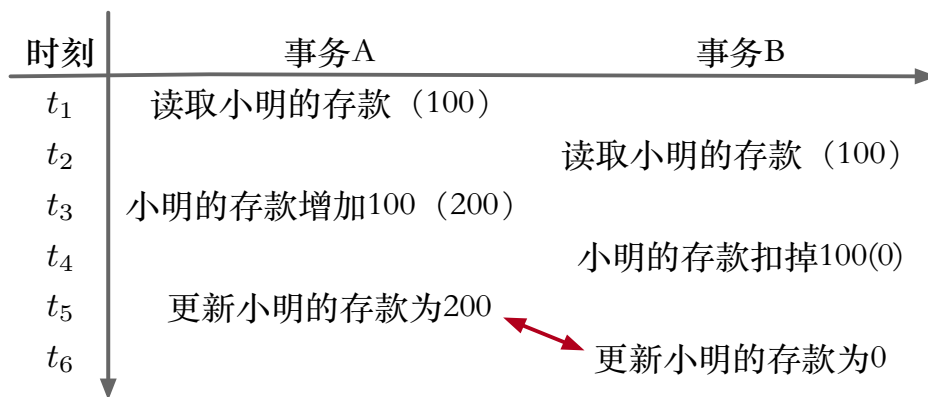
1、并发事务处理技术

□ 银行场景中新来了两个事务，事务A先被请求：

- 事务A，小明发工资100元。
- 事务B，家长用小明银行卡消费了100元。

□ 事务B与事务A在并发过程中存在写写冲突：导致脏写 (dirty write)

- 两个事务串行执行，正确的结果是小明的存款余额为100元。
- 事务A尝试更新余额为200元
- 事务B尝试更新余额为0元



隔离性-写写冲突



1、并发事务处理技术

□ 为了更好地解释并发控制过程中，数据库对事务的处理流程，首先引入一个概念：调度。

□ 调度为事务的并发过程中，决定事务中每个操作的执行顺序。

- 读操作read(X)和写操作write(X)分别表示从数据库中读取记录X的数据和将数据写入记录X中。
- 使用读操作和写操作来描述事务的执行过程。
- 调度为读操作和写操作组成的序列。
 - 更新操作可以被替换为3个操作。

```
UPDATE account SET balance = balance - 100.00 WHERE name = '小明';
```

```
read(小明的余额) ;  
小明的余额 = 小明的余额 - 100;  
write(小明的余额) ;
```

调度



1、并发事务处理技术

□ 串行调度(serial schedule): 事务串行执行

□ 调度定义下的并发控制:

- 给定一个并发调度 S , 存在一个串行调度 S' , 在任何数据库状态下, 按照调度 S 和调度 S' 执行后所产生的结果都是相同的。
- 此时调度 S 被称之为可串行化调度 (serializable schedule) 。

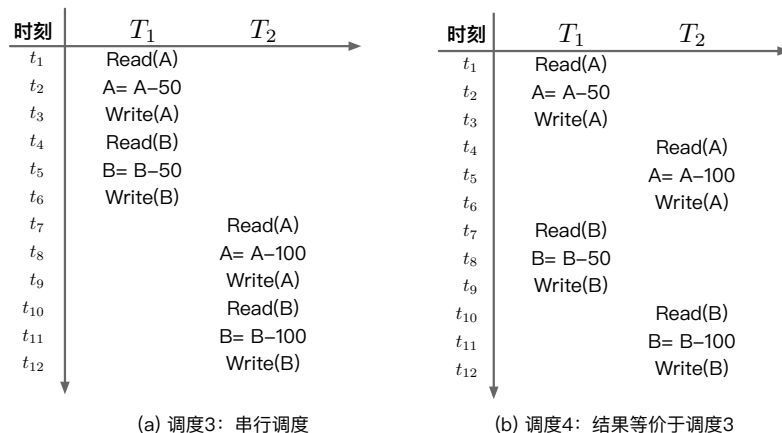
□ 可串行化调度的数量十分巨大, 且难以校验, 数据库中一般通过找到可串行化调度的子集 (充分条件), 即找到能够提前确认是可串行调度的并发调度, 进而提升调度效率。

可串行化调度

1、并发事务处理技术

□ 场景:

- 小明仍然在超市进行了两笔消费，分别为100元和200元。小明有超市的会员卡，可以抵扣一半的金额。
 - 小明的存款用A表示，初始为300。
 - 会员卡的数据用B表示，初始为300。
 - 两笔消费分别为A减去50，B减去50(事务 T_1)；
 - A减去100，B减去100 (事务 T_2)。
- 调度3描述的是该事务的串行调度。
- 调度4则描述了一种可能的并发调度。
- 调度4在任何情况下与调度3都能产生相同的结果。是一种可串行化调度



可串行化调度



1、并发事务处理技术

□ **定义：冲突可串行化调度**，是从冲突的角度出发，针对一个调度 S 去发现其等价的串行调度 S' 来确定 S 是一个可串行化调度。

- **一次操作交换**定义为交换事务调度序列中相邻的两个操作，一个交换操作可以将一个调度 A 变成另外一个调度 B 。
- **当交换不会影响两个调度的一致性时**，定义该交换得到的两个调度**是等价的**，该交换为等价交换。
 - 等价操作
 - 交换连续两个相同数据读取操作的顺序： $R_{T1}(A) R_{T2}(A)$
 - 交换连续两个不同数据读写操作的顺序： $R_{T1}(A) W_{T2}(B); W_{T1}(A) R_{T2}(B); W_{T1}(A) W_{T2}(B)$
 - 非等价操作
 - 交换连续两个相同数据的读写、写写操作： $R_{T1}(A) W_{T2}(A); W_{T1}(A) R_{T2}(A); W_{T1}(A) W_{T2}(A)$

冲突可串行化调度



1、并发事务处理技术

R(A)	W(A)		R(B)	W(B)			
		R(A)	W(A)			R(B)	W(B)

R(A)	W(A)		R(B)	W(B)			
		R(A)		W(A)		R(B)	W(B)

R(A)	W(A)		R(B)	W(B)			
		R(A)			W(A)	R(B)	W(B)

R(A)	W(A)	R(B)		W(B)			
			R(A)		W(A)	R(B)	W(B)

R(A)	W(A)	R(B)	W(B)				
				R(A)	W(A)	R(B)	W(B)

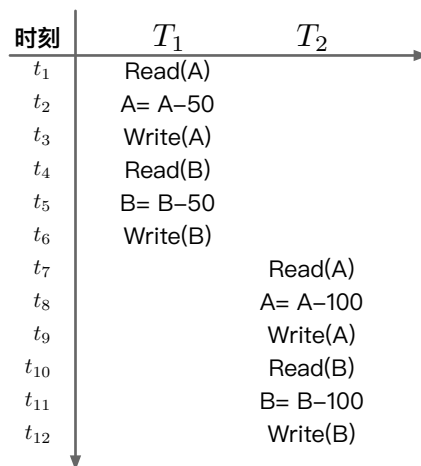
冲突可串行化调度



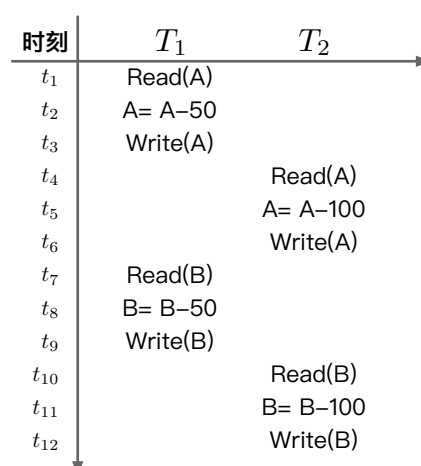
1、并发事务处理技术

□ 冲突可串行化调度验证的原则：

- 当并发的两个事务之间不存在冲突时，可以很容易的通过交换操作将当前调度 S' 转化成等价的串行调度。
- 对于存在冲突的并发事务调度 S' ，在保证冲突操作的次序不变的前提下，通过仅交换不冲突的操作而转化为等价的串行调度 S 时，称调度 S' 和调度 S 冲突等价，调度 S' 为冲突可串行化调度。



(a) 调度3：串行调度



(b) 调度4：结果等价于调度3

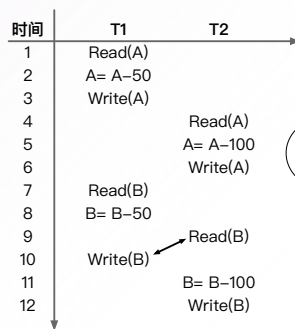
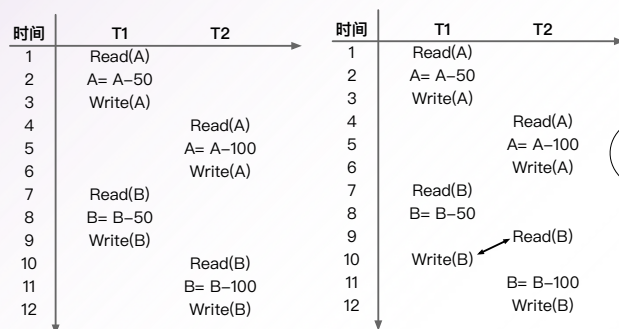
冲突可串行化调度验证

1、并发事务处理技术

□ 优先图：利用冲突关系，可以将并发的事务构建成一个图来验证事务的并发调度 S' 是否是冲突可串行化调度。

- 若 T_i 与事务 T_j 存在冲突（读写冲突、写写冲突、写读冲突），且 T_i 的冲突操作在 T_j 冲突操作之前，则 T_i 向 T_j 连一条有向边 (T_i, T_j) 。

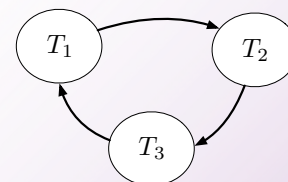
□ 冲突可串行化的充分必要条件：优先图无环



(a) 调度4的优先图



(b) 调度4.4的优先图



(c) 一个复杂的非冲突可串行的优先图

冲突可串行化调度验证



1、并发事务处理技术

□ 锁：数据项的访问权限

- 只有持有锁才可访问数据项

□ 锁的类型

- 共享锁 (shared lock, S锁) :
 - 如果某事务在数据项上加了共享锁,则该事务只能读取该数据项但不能对其进行修改
 - 此时其他事务可在该数据项上加上共享锁,但不能加互斥锁
- 互斥锁 (exclusive lock, X锁)
 - 如果某事务在数据项上加了互斥锁, 则该事务既能读取该数据项也能对其进行修改
 - 此时其他事务既不能在该数据项上加共享锁, 也不能加互斥锁

锁



1、并发事务处理技术

- 锁的相容性矩阵, T_1 持有锁, T_2 请求锁
- 是否可获得 (即相容)

T_2 T_1	共享锁	互斥锁
共享锁	相容	不相容
互斥锁	不相容	不相容



1、并发事务处理技术

事务 T1: R(A) W(A) R(B) W(B)
T2: R(A) R(B)

	T ₁	T ₂
1	Read(A)	
2		Read(A)
3	A=A+1	
4	Write(A)	
5		Read(B)
6	Read(B)	
7		A+B
8		Commit
9	B=B-1	
10	Write(B)	
11	Commit	

调度 T1: R(A) W(A) R(B) W(B)
T2: R(A) R(B)

	T ₁	T ₂
1	X-Lock(A)	
2	Read(A)	
3		S-Lock(A)
4	Write(A)	
5	X-Lock(B)	
6	Read(B)	
7	Write(B)	
8	Commit	
9	Unlock(A)	
10	Unlock(B)	Read(A)
11		S-Lock(B)
12		Read(B)
13		Commit
14		Unlock(A)
15		Unlock(B)

可串行化

也可恢复

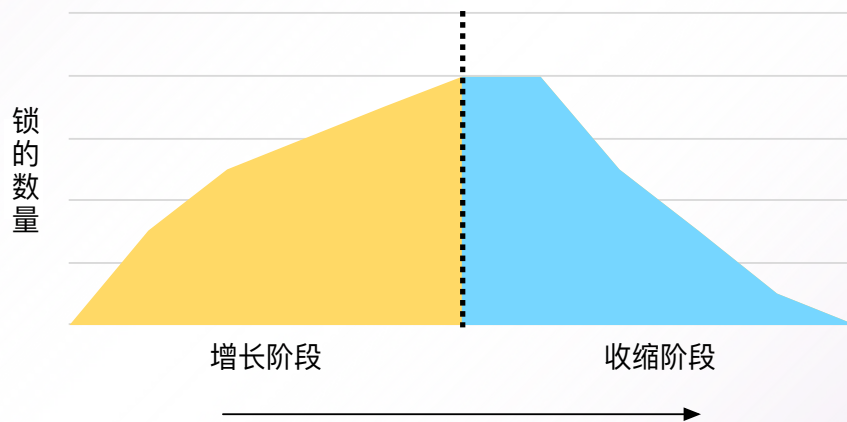
两阶段锁



1、并发事务处理技术

□ 事务的两个阶段:

- 增长阶段 (growing phase)
 - 事务可以获得所需的锁, 但不能释放锁
- 收缩阶段 (shrinking phase)
 - 事务可以释放已获得的锁, 但不能申请新的锁



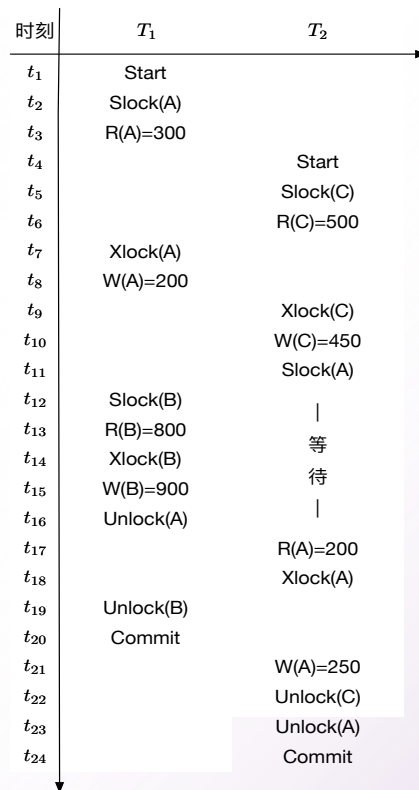
两阶段锁



1、并发事务处理技术

□ 示例：

- 事务 T_1 : 小明转账100元给小刚
- 事务 T_2 : 小红转账50元给小明
- A 、 B 、 C 分别表示小明、小刚、小红的余额
- R 和 W 分别表示读和写操作
- 时刻 t_{12} - t_{16} : T_2 等待 T_1 完成操作并释放

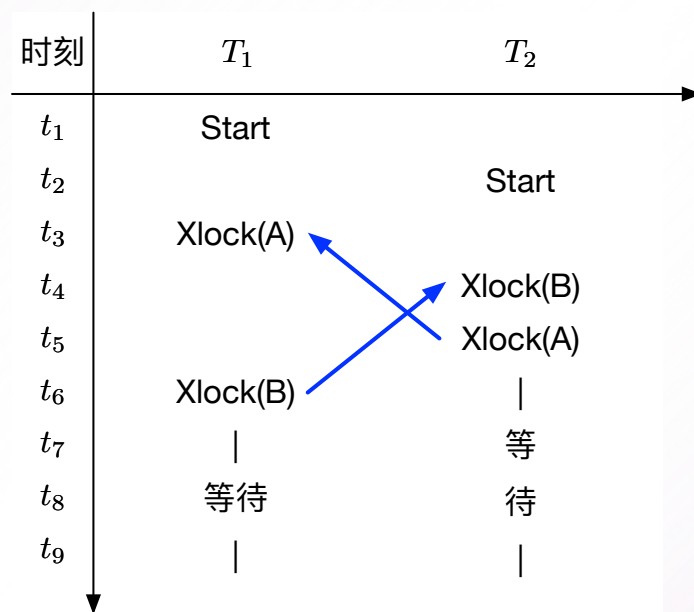


两阶段锁

1、并发事务处理技术

□ 死锁

- 事务彼此都持有锁又同时在等待对方释放所需要的锁
- 不能继续执行并因此陷入了持续的等待状态



死锁



1、并发事务处理技术

□ 假设有事务 T_1 和事务 T_2 , T_1 的时间戳更旧。

□ Wait-Die (旧事务 T_1 等待新事务 T_2)

- 如果事务 T_2 持有锁, 事务 T_1 申请加锁, 那么 T_1 等待 T_2 。
- 如果事务 T_1 持有锁, 事务 T_2 申请加锁, 那么 T_2 被撤销并重启, 重启后 T_2 的时间戳仍为原来的时间戳。

□ Wound-Wait (新事务 T_2 等待旧事务 T_1)

- 如果事务 T_2 持有锁, 事务 T_1 申请加锁, 那么 T_2 被撤销并重启, 同时释放持有的锁。重启后 T_2 的时间戳仍为原来的时间戳;
- 如果事务 T_1 持有锁, 事务 T_2 申请加锁, 那么 T_2 等待 T_1 。

死锁解决方案



1、并发事务处理技术

- 多版本并发控制：对于一个数据项，保存多个物理版本，供不同的事务使用
- 前面介绍的方法通过时间复用实现并发控制，多版本机制通过空间复用实现高效并发控制
- 基本原则：
 - 事务对数据项进行写操作时，产生该数据项的一个新版本
 - 事务对数据项进行读操作时，读取事务开始时该数据项的最新版本
- 优点：
 - 事务的读操作无需等待其他事务的写操作
 - 事务的写操作无需等待其他事务的读操作

MVCC



1、并发事务处理技术

数据项A有多个版本，表示小明的银行卡余额。每个版本的数据内容不同，且有效的开始时间和结束时间前后连续。最开始，小明有1500元，随后通过勤工俭学收入了300元变为1800元，然后他又花了50元充值网费，还剩1750元

版本	内容	时间戳
A_0	1500	0
A_1	1800	10
A_2	1750	11

MVCC



1、并发事务处理技术

- 对时间戳排序协议进行多版本化的拓展
- 隔离级别：可串行化
- 每个事务 T_i 在开始执行前获得唯一的时间戳 $TS(T_i)$
- 每个数据项 x 都有多个物理版本，用序列 $X_0, X_1, \dots, X_m$ 表示。每个版本 X_i 由三个数据组成：
 - 该版本的值
 - $W_TS(X_i)$ ：产生该版本的事务时间戳
 - $R_TS(X_i)$ ：成功读取该版本的事务的最大时间戳

MVCC



1、并发事务处理技术

- 对于只读事务，根据事务的时间戳来读取事务开始时的最新版本，不需要任何锁机制
- 对于更新事务，多版本两阶段锁协议有很多变种，介绍其中两种典型方法
- 方法一：读操作不加锁，写操作加严格两阶段锁。对于事务 T_i 和数据项 X ，设 X_j 为时间戳不超过 $TS(T_i)$ 的最大的版本，即事务 T_i 开始时的最新版本。
 - 如果事务 T_i 对 X 只进行读操作，则数据库返回 X_j 的值
 - 如果事务 T_i 对 X 进行写操作，为 X 创建一个新版本，并对其加互斥锁，即同时只能创建一个新版本。如果事务 T_i 成功提交，则将该新版本时间戳设置为 $TS(T_i)$ ；如果事务回滚，删除该新版本

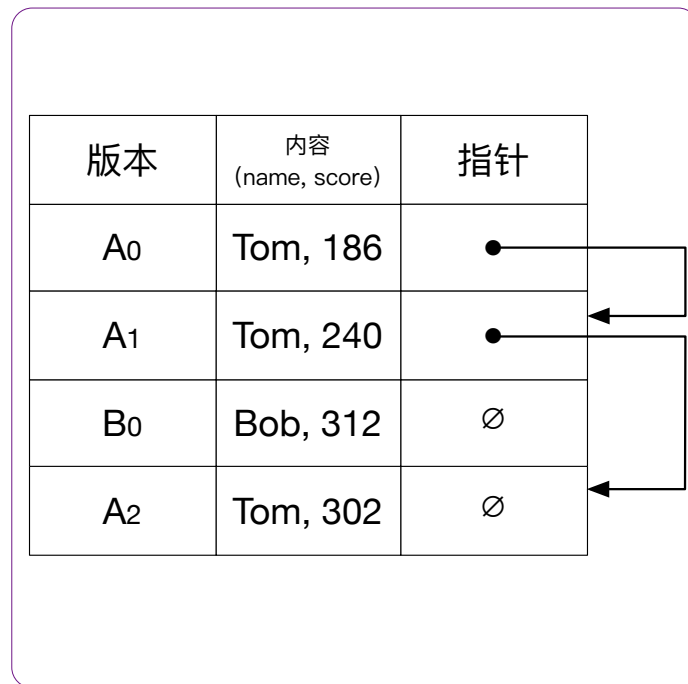
MVCC



1、并发事务处理技术

□ 仅追加存储

- 所有版本存储在一个表空间中，插入版本链表中
- 组织版本链表的两种顺序：
 - 从旧到新：新版本追加到链表尾部；查找新版本时需要遍历链表。索引指针始终指向第一个版本，不需要更新索引指针
 - 从新到旧：新版本插入链表头部，同时更新索引指针；查找新版本时不需要遍历链表
- **缺点：**将多版本信息存储在原数据空间中，数据一直增长。淘汰老版本数据时，需要遍历数据，造成性能不平稳

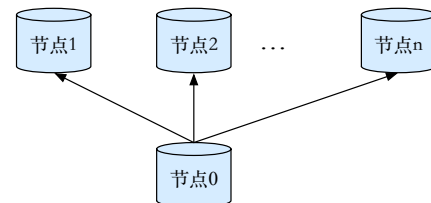


MVCC

2、并发查询处理技术

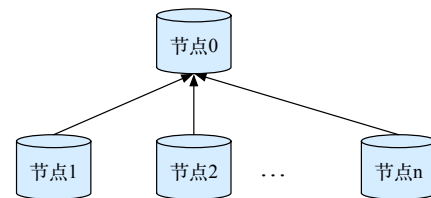
□广播

- 由源节点向其他各节点广播需要的数据



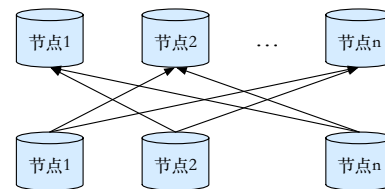
□聚集

- 各节点将数据统一发送到协调节点



□重分配

- 每个节点将数据发送至其他所有节点



网络传播算子



2、并发查询处理技术

□两张小表

- 如果非分布式存储，可以直接连接
- 如果是分布式存储，则先聚集再连接

□一张大表和一张小表

- 将小表广播，各节点分别与本地大表数据连接

□两张大表

- 如果分区策略相同且连接键恰为分区键，则各节点可以在本地进行连接，然后将结果汇总至协调节点
- 否则需要重分配

分布式查询优化



2、并发查询处理技术

□问题

- 数据量超出单台机器存储容量上限

□解决方法

- 将数据分片，每个节点只存储部分数据

□分片类型

- 水平分片：按行切分，不同的行存储于不同节点
- 垂直分片：按列切分，不同的列存储于不同节点

□水平分片算法

- 哈希分片，范围分片，列表分片

数据分片



2、并发查询处理技术

□ 哈希分片

- 优点：数据分布均匀
- 缺点：不支持范围查询

节点0

年份	省份
2018	黑龙江
2020	上海
2022	辽宁

节点1

年份	省份
2017	江苏
2019	吉林
2021	浙江

□ 范围分片

- 优点：实现简单
- 缺点：数据分布可能不均

节点0

年份	省份
2017	江苏
2018	黑龙江
2019	吉林

节点1

年份	省份
2020	上海
2021	浙江
2022	辽宁

□ 列表分片

- 主要针对非数值型属性
- 优缺点与范围分片相似

节点0

年份	省份
2017	江苏
2020	上海
2021	浙江

节点1

年份	省份
2018	黑龙江
2019	吉林
2022	辽宁

数据分片

2、并发查询处理技术

□传统哈希的问题：

- 节点数目增删引起大规模数据移动

数据编号	哈希值	节点号
1	132	0
2	257	2
3	978	0
4	245	2
5	643	1



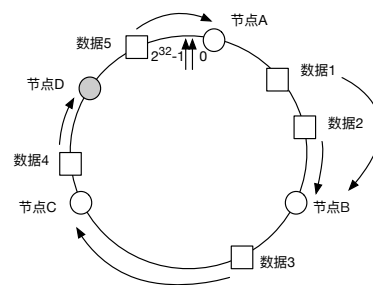
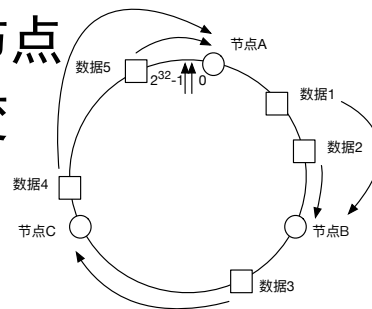
数据编号	哈希值	节点号
1	132	0
2	257	1
3	978	2
4	245	1
5	643	3

$$\text{node} = \text{hash}(\text{data}) \% 3$$

$$\text{node} = \text{hash}(\text{data}) \% 4$$

□一致性哈希

- 将节点和数据映射到哈希环
- 根据哈希环确定数据存放在哪个节点
- 节点数目变化时，尽可能小地改变数据与节点之间的对应关系

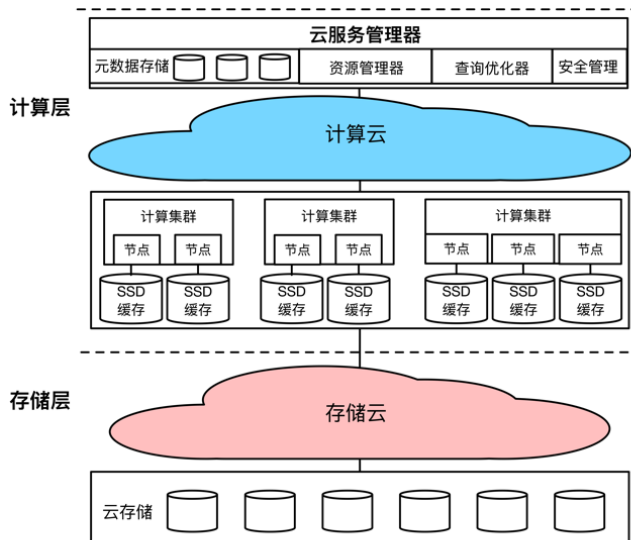


数据分片

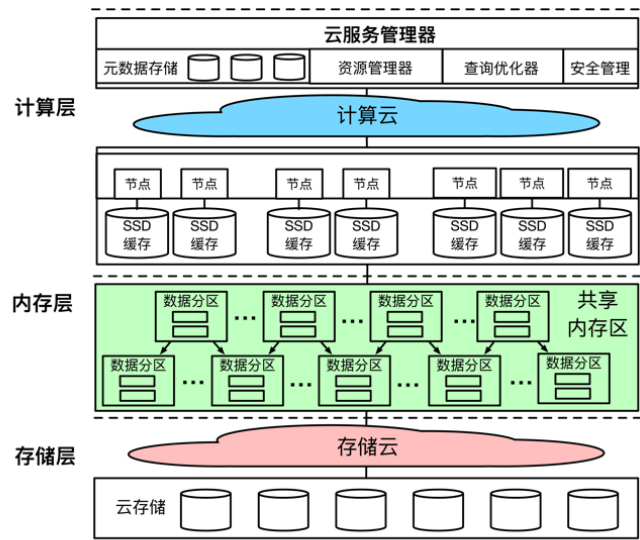
2、并发查询处理技术

• OLAP云原生数据库架构

- 采用计算与存储分离的架构来提高系统弹性与高可用性
- 存储云节点按照数据分片进行管理，保证了数据分析过程的高吞吐率
- 上层计算云节点不区分主和备节点，计算集群中的工作节点独立地处理各个数据分片的分析任务



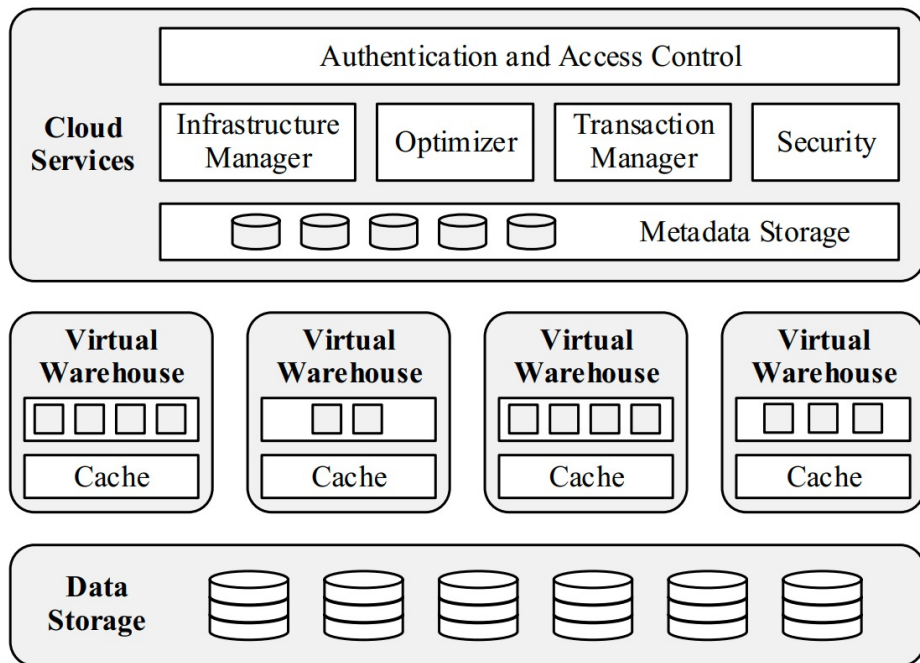
(a) 云原生OLAP的计算-存储分离架构



(b) 云原生OLAP的计算-内存-存储分离架构



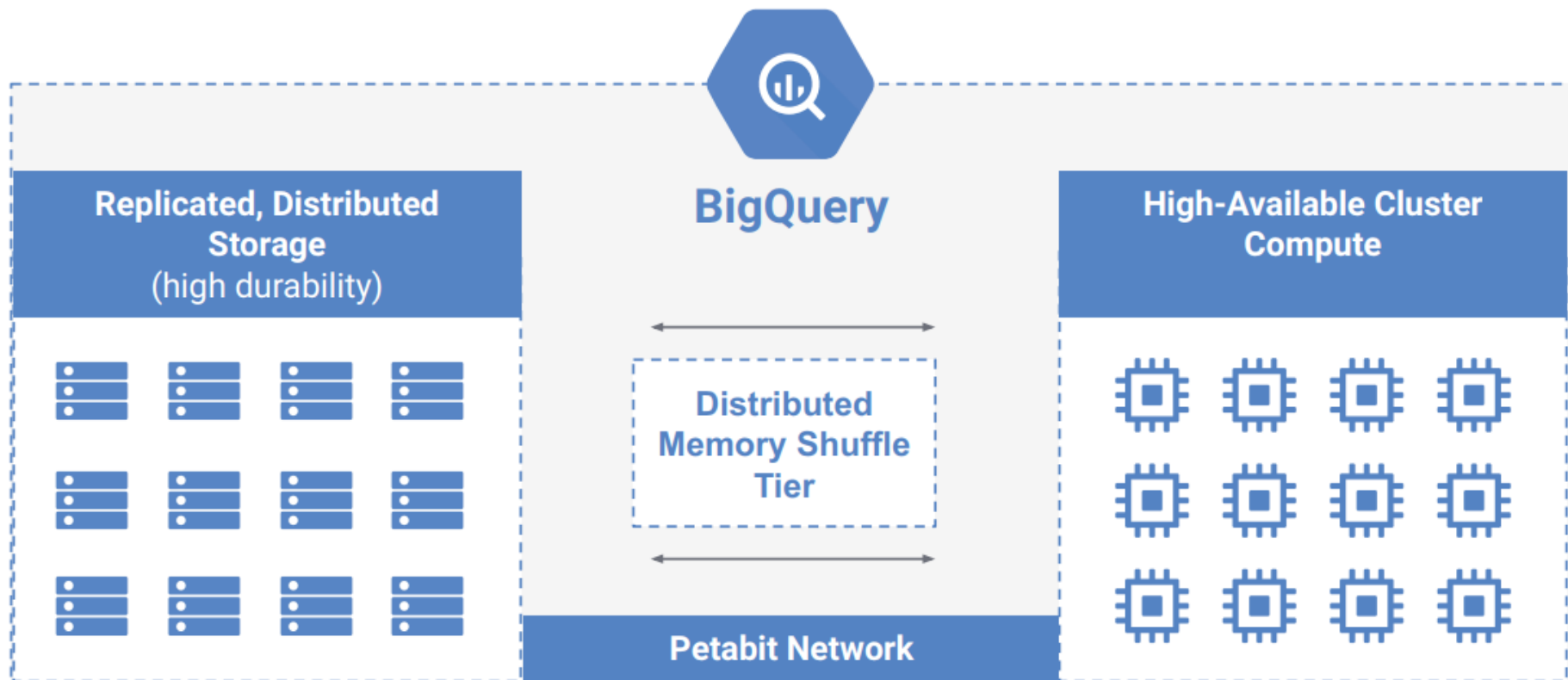
2、并发查询处理技术



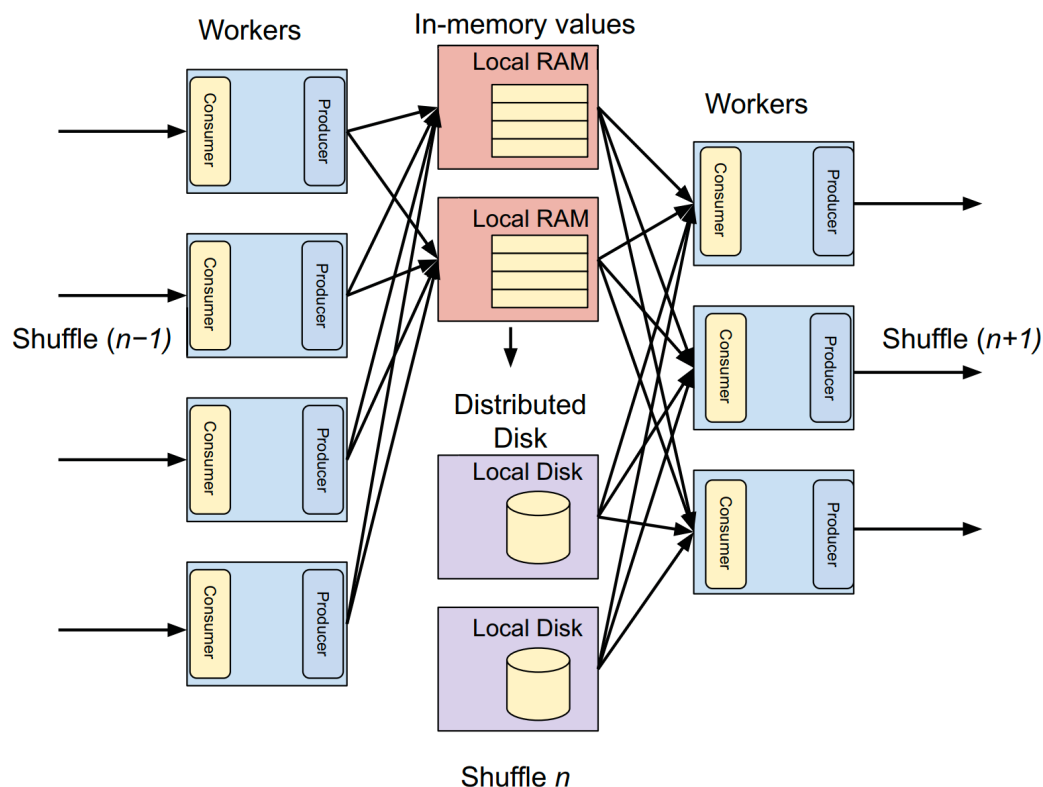
■ 关键特性:

- 计算-存储分离
- 统一元数据管理
- 统一查询调度与优化
- 集群相互隔离，服务单租户
- 本地存储集群
- 在线动态扩容
- 向量化执行引擎
- 云存储计算下推

2、并发查询处理技术



2、并发查询处理技术

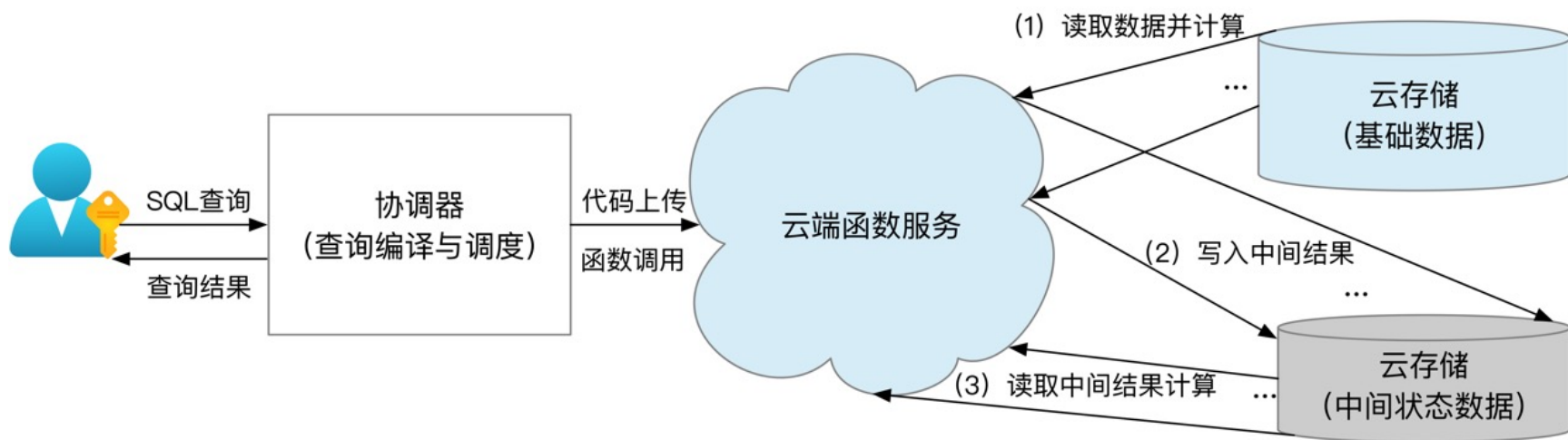


■ 计算模型：

- 生产者消费者异步执行模式
- 生产者产生数据分区
- 消费者合并与聚集
- 分布式内存Shuffle
- 溢出分布式存储
- 在线动态扩容
- 动态变更并发度

2、并发查询处理技术

- 主要方法：基于云函数的无服务器计算，通过即时启动多个函数来按需执行查询。
- 挑战：（1）函数有生命周期，状态需要保存；（2）存在木桶效应
- 解决方案：以云存储保存函数的中间状态；启动重复任务执行部分操作





2、并发查询处理技术

Auto-WLM 是什么？

- Amazon Redshift 正在使用的基于机器学习的自动负载管理器。
- 对查询进行智能调度以提高吞吐量，同时支持在负载激增时横向扩展集群。
- 目前 Amazon Redshift 的所有查询都由 Auto-WLM 调度，后者每天进行数百万次决策。

设计挑战

- 查询类型多样
 - ★ 读/写、长/短、本地/联邦，不同查询有不同的资源需求。
- 负载变化难以预测
 - ★ 例如突发的 ad-hoc 请求将会引起集群负载激增。
- 决策时间受限
 - ★ 为避免对短查询的性能产生显著影响，调度决策时间不应过长。



2、并发查询处理技术

设计概览

- 查询性能模型 (ML Predictor)
 - ★ 负责预测查询延迟和所需内存。
- 查询优先级策略 (Priority Assigner)
 - ★ 根据模型输出为查询分配优先级，通常是短任务优先。
- 准入控制器 (Admission Controller)
 - ★ 决定到来查询的去向：在主集群执行/在并发集群执行/进入短队列等待/进入用户队列等待。
- 利用率监视器 (Utilization Monitor)
 - ★ 计算集群资源的利用率，提示准入控制模块应该放行更多或更少的查询。
- 机器学习训练器 (ML Trainer)
 - ★ 负责维护训练数据的滑动窗口并且定期更新查询性能模型。

2、并发查询处理技术

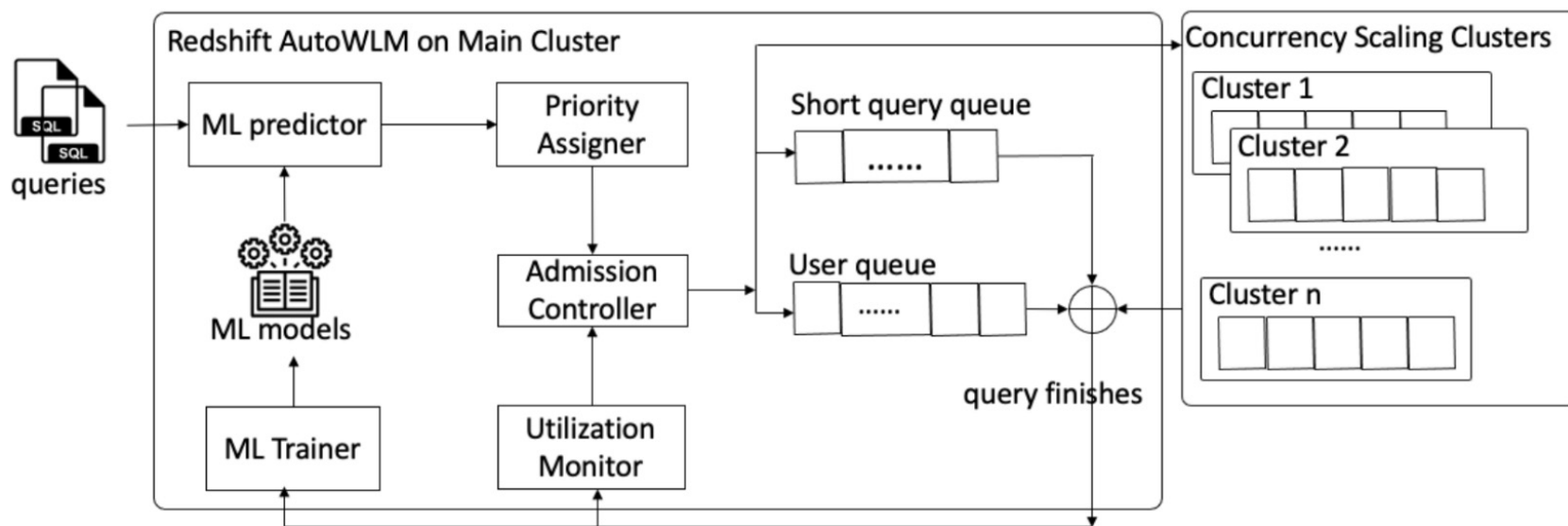


Figure 2: AutoWLM architecture overview. When a query arrives, the ML predictor determines the resource requirements of the query. The priority assigner determines how short the incoming query is to support shortest-job-first scheduling. Given the priority, the admission controller decides if the query should be executed on the main's clusters short query queue, user queue, or sent to a concurrency scaling cluster. Once a query finishes, two things happen: (1) the ML trainer adds the observed query plan and latency to the training set, and may update the ML predictor, and (2) the utilization monitor determines if the cluster's resources are under- or over-subscribed, telling the admission controller to let in more/fewer queries.

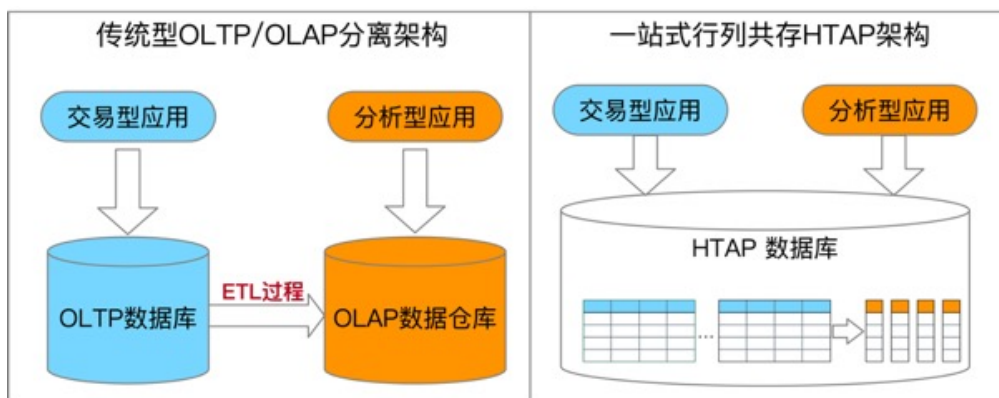
3、混合事务与查询处理技术

➤ HTAP: Hybrid Transactional Analytical Processing (混合事务与分析处理)

- 统一支持OLTP和OLAP场景
- 常用场景：在线电商平台、金融反欺诈系统等

➤ HTAP相比于传统OLTP与OLAP技术的优势

- 更简单的数据准备过程：避免了耗时的ETL操作
- 更快的数据处理与分析：使OLTP与OLAP负载都可以被快速执行与响应
- 更高效的数据管理方式：使用统一的系统管理数据



HTAP的相关应用

- **Gartner**预测在**2024**前，**HTAP**技术将实时数据分析型应用领域被广泛采用
- **HTAP**技术适用于需要同时保证高性能的事务处理与高度实时性的数据分析的应用
- **例如**，在电子商务应用中需要在线分析销售趋势的场景；在欺诈识别应用中需要在处理不同客户事务的同时识别出欺诈行为



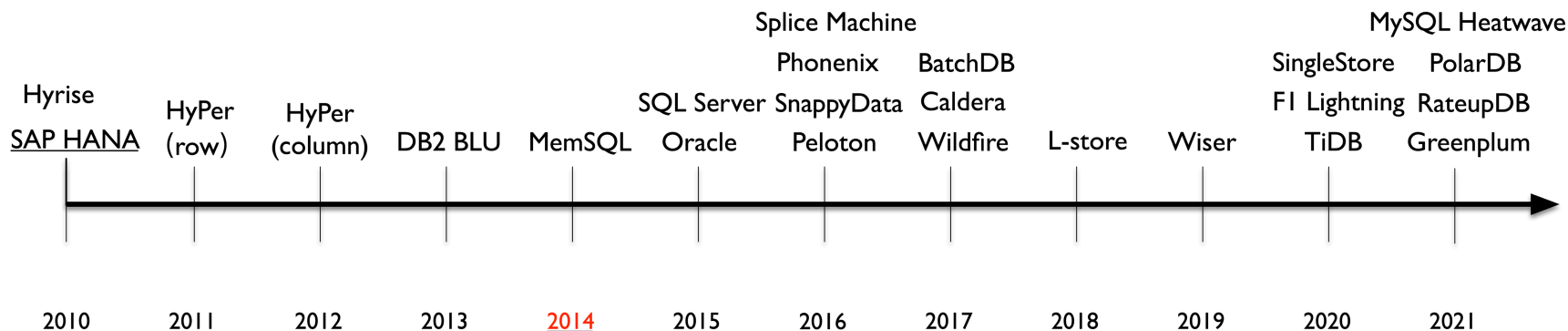


HTAP数据库的发展历程

□ 在过去的十多年里, 已经出现了许多的**HTAP数据库**

□ 总体来说, HTAP数据库的发展历程可分为以下三个阶段:

- **第一阶段 (2010-2014):** HTAP数据库主要采取基于单机的主列存储
- **第二阶段 (2014-2020):** HTAP数据库主要在单机行存储的基础上扩展列存储
- **第三阶段 (2020-目前):** HTAP数据库主要在分布式行存储的基础上扩展列存储



行列共存型HTAP数据库

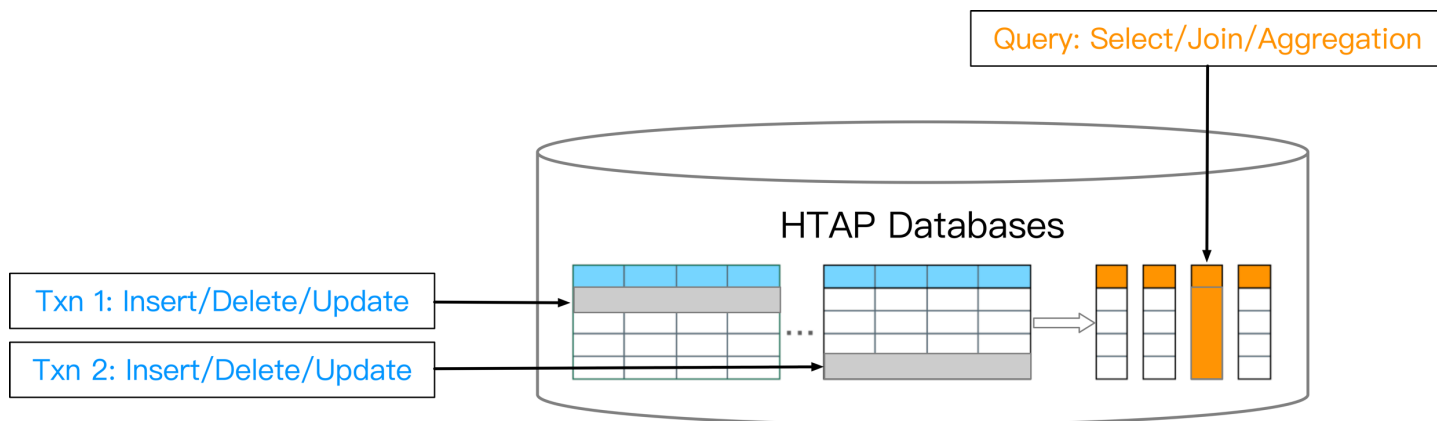
□ 经验法则1: 行存储更适合于OLTP型负载

- OLTP负载通常为面向行的、频繁更新的、短时延的事务型操作

□ 经验法则2: 列存储更适合于OLAP型负载

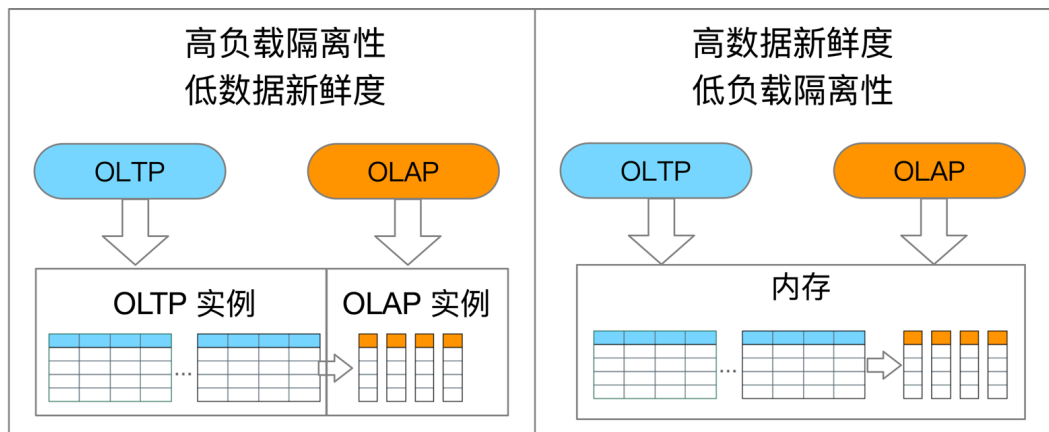
- OLAP负载通常为面向列的、只读的、复杂分析型查询

□ 目前主流HTAP数据库基本都采用行列共存的数据组织方式



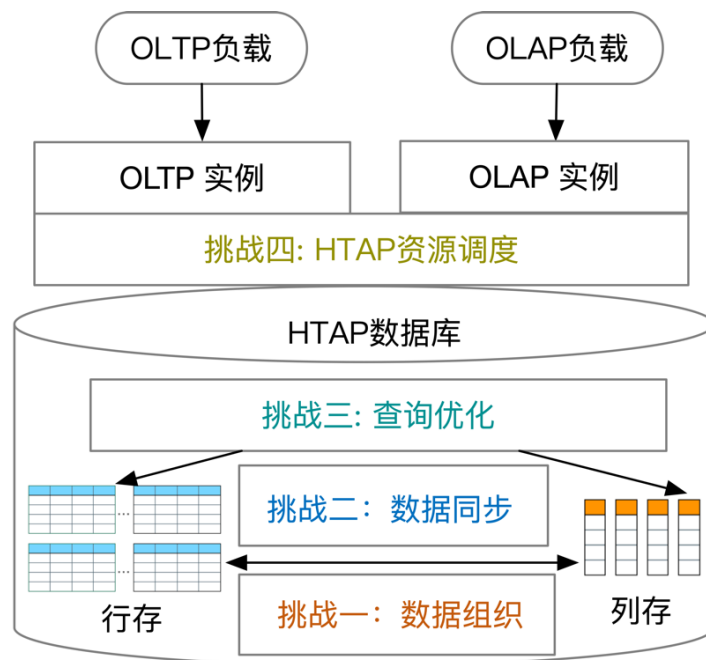
行列共存型HTAP数据库

- 负载隔离性: 处理混合负载（OLTP&OLAP）的隔离程度
- 数据新鲜度: OLAP分析的最近元组数目与OLTP所有更新的元组数目的比率
- 关于负载隔离与数据新鲜度的权衡
 - 高负载隔离性导致低数据新鲜度
 - 高数据新鲜度导致低负载隔离



HTAP数据库面临的主要挑战

- ❑ **挑战一(数据组织):** 如何根据以往的 HTAP 负载以适应性地组织数据以优化系统的处理性能与降低存储代价
- ❑ **挑战二 (数据同步):** 如何将最新的事务数据更新到列存储中以保证数据高新鲜度并同时尽量减小数据同步对系统造成的性能影响
- ❑ **挑战三 (查询优化):** 如何为查询的算子在庞大的候选空间中选择最优地存储访问路径(行存或列存)
- ❑ **挑战四 (HTAP资源调度):** 如何合理地系统中执行 OLTP 负载的实例和执行 OLAP 负载的实例分配 CPU、内存等资源,使得系统的性能与数据新鲜度最大化



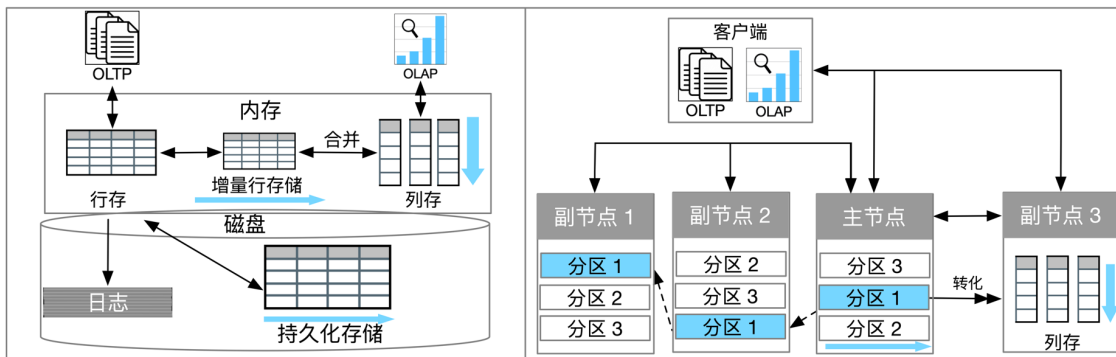
HTAP数据库的主要架构

(a)主行存与内存型列存

(b)分布式行存与列存副本

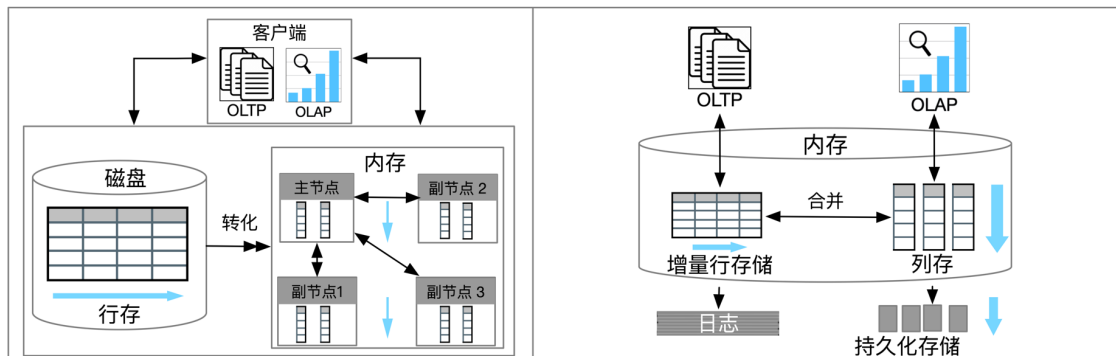
(c)磁盘型行存与分布式列存

(d)主列存与增量型行存



(a) 主行存与内存型列存

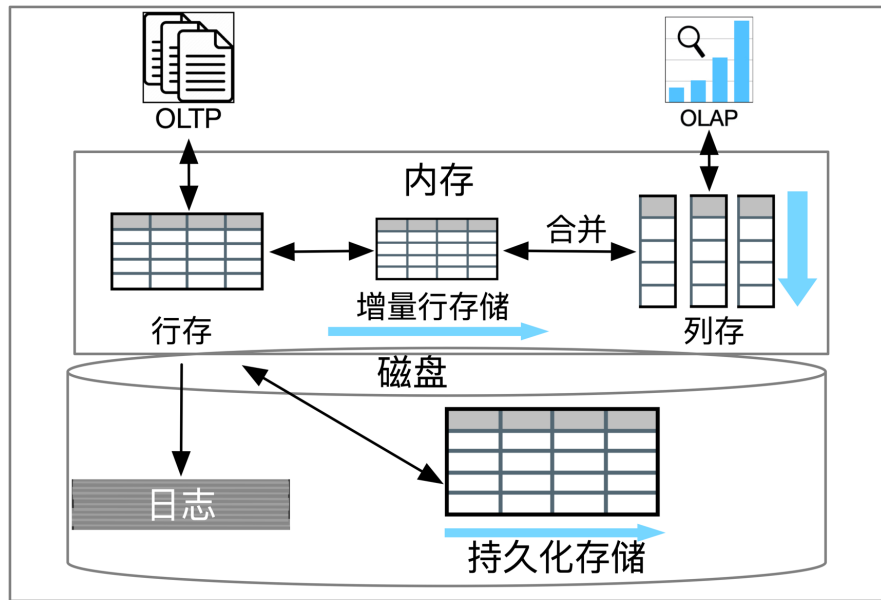
(b) 分布式行存与列存副本



(c) 磁盘型行存与分布式列存

(d) 主列存与增量型行存

(a) 主行存与内存型列存



(a) 主行存与内存型列存

优点:

OLTP吞吐量**高**

OLAP吞吐量**高**

数据新鲜度**高**

缺点:

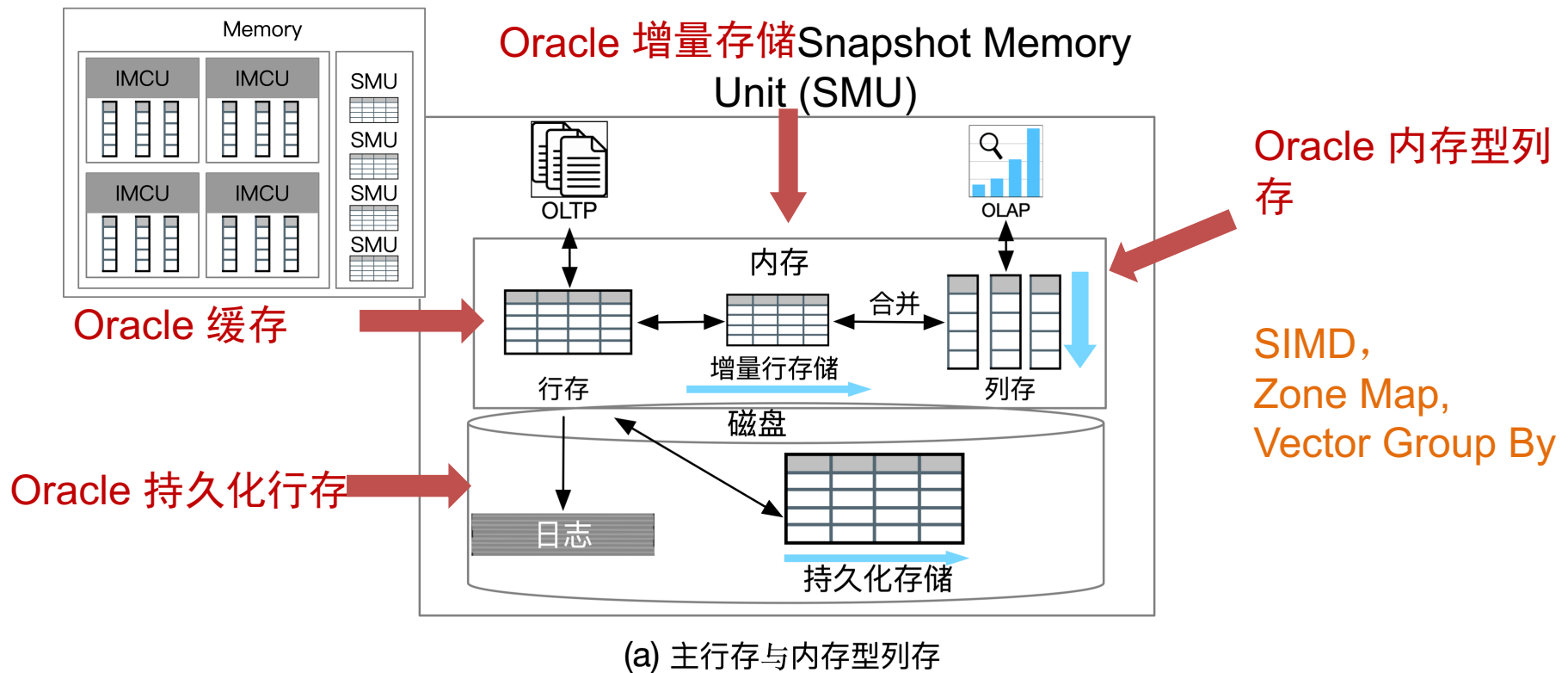
OLAP扩展性**低**

负载隔离性**低**

应用场景: 高吞吐量与低扩展性应用

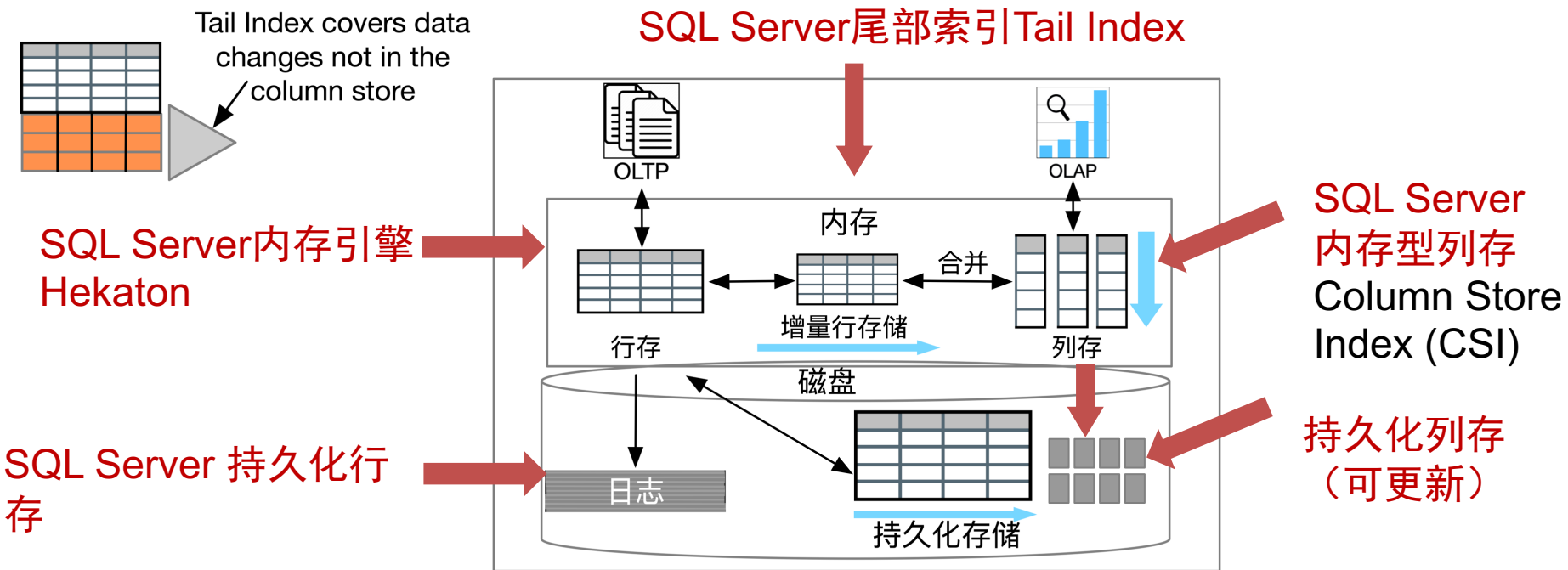
如**银行金融业** (事务处理与实时报表)

(a)类架构系统: Oracle Dual-Format



Lahiri, Tirthankar, et al. "Oracle database in-memory: A dual format in-memory database." In ICDE, 2015.

(a)类架构系统: SQL Server



(a) 主行存与内存型列存

Larson, Per-Åke, et al. "Real-time analytical processing with SQL server." *PVLDB* 8(12), 2015: 1740-1751.



基于(a)类架构的HTAP数据库比较

HTAP 数据库	基于行存的OLTP	增量存储 (Delta)	基于列存的 OLAP	可持久化列存
Oracle Dual-Format	基于内存优化的 的行存 (Cache)	Snapshot Metadata Unit(SMU)	In Memory Column Store (IMCS)	否
SQL Server	基于内存优化的 的行存 (Hekaton)	尾部索引 Tail index	Column Store Index (CSI)	是

(b) 分布式行存与列存副本

优点：

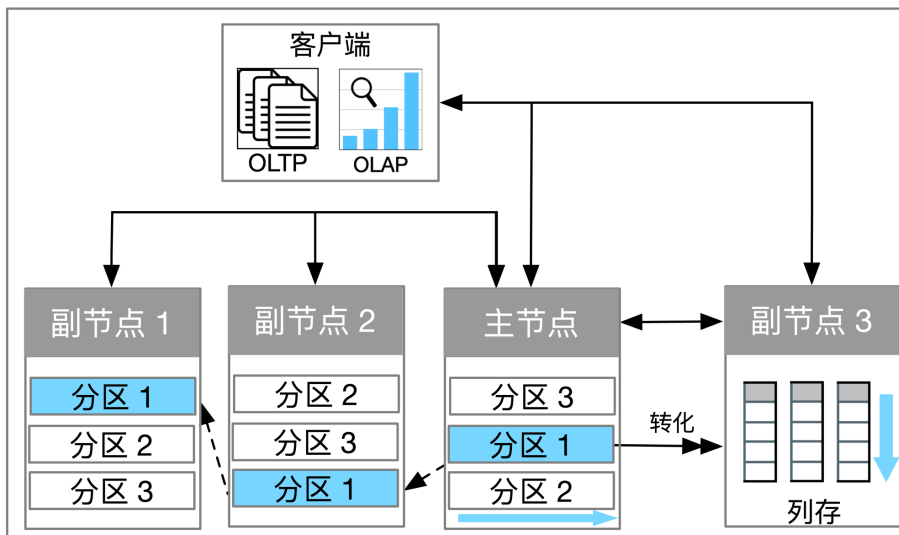
负载隔离性**高**

系统扩展性**高**

缺点：

数据新鲜度**低**

应用场景：高扩展性与秒级分析时延
如**大型电子事务**（事务处理与T+0报表）

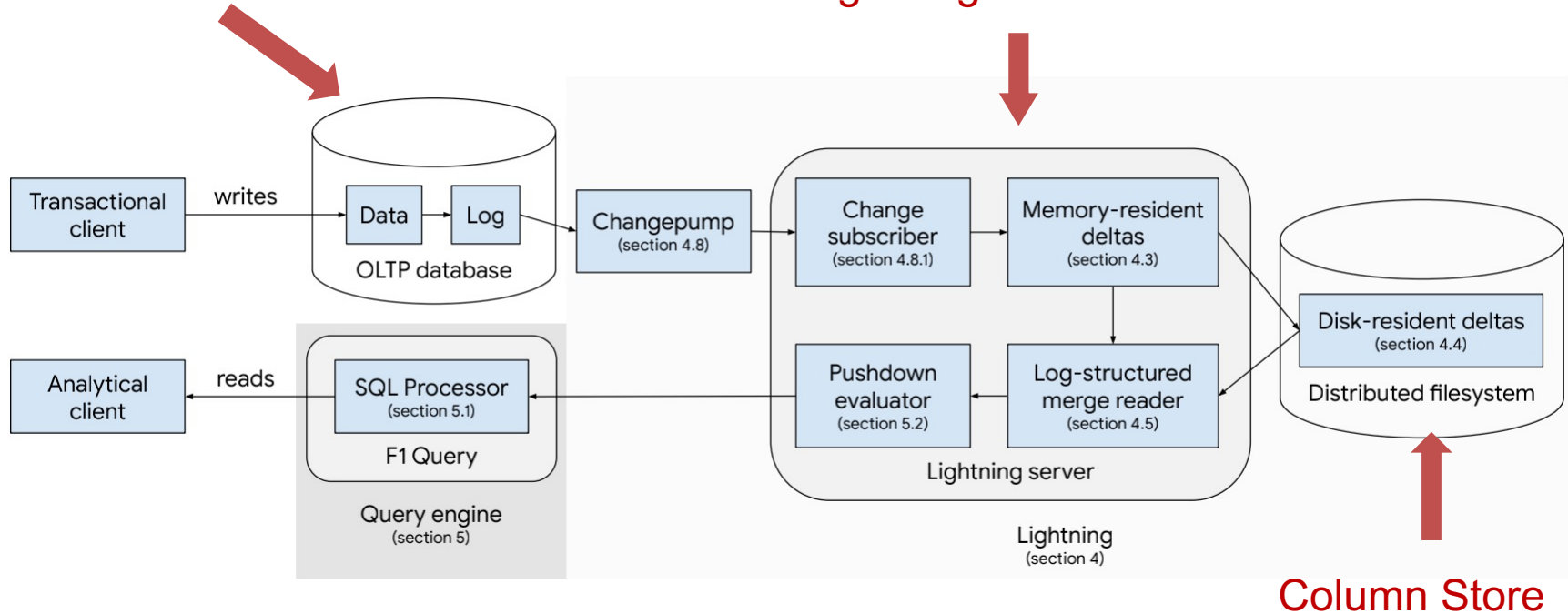


(b) 分布式行存与列存副本

(b)类架构系统: F1 Lightning

Spanner-based Row Cluster

Lightning Server for delta store

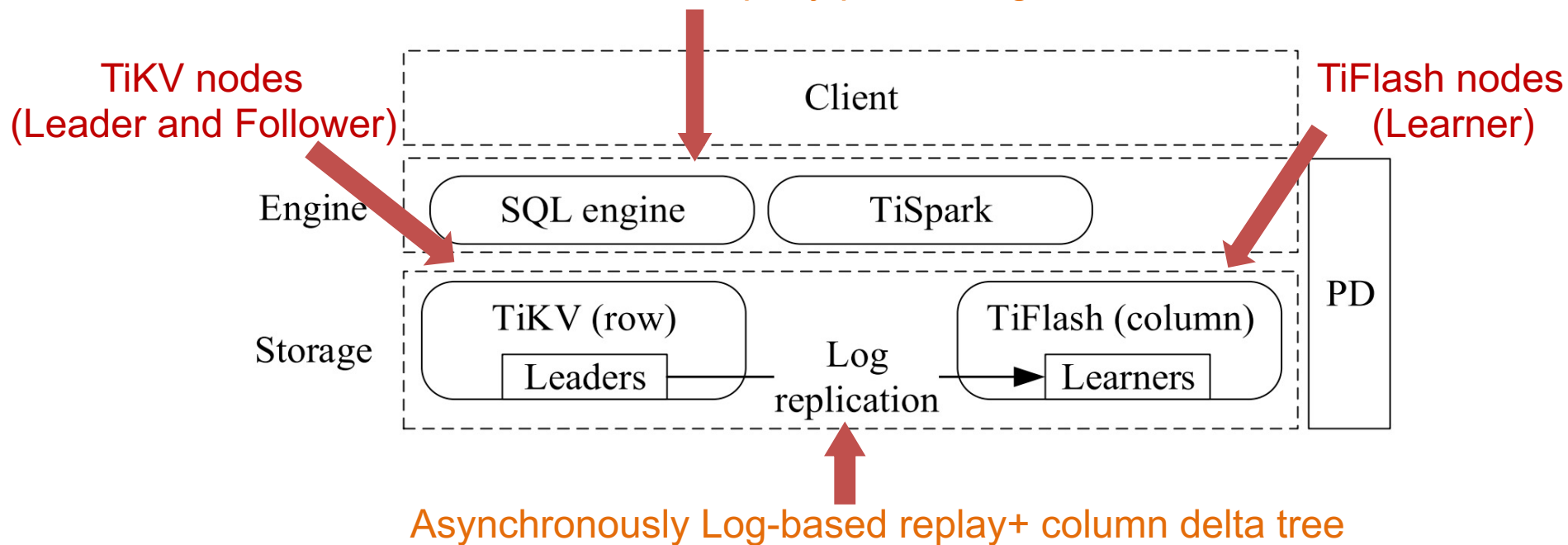


Yang, Jiacheng, et al. "F1 Lightning: HTAP as a Service." *PVLDB* 13(12), 2020: 3313-3325.

(b)类架构系统: TiDB

2PC+Raft for TP

Cost-based query processing for AP



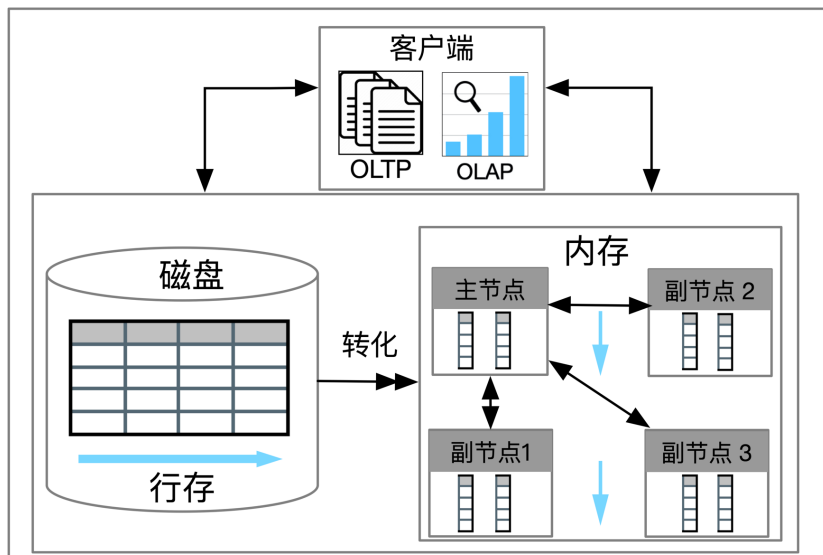
Huang, Dongxu, et al. "TiDB: a Raft-based HTAP database." *PVLDB* 13(12), 2020: 3072-3084.



基于(b)类架构的HTAP数据库比较

HTAP 数据库	基于行存的OLTP	分布式协议	增量存储 (Delta)	基于列存的OLAP
F1 Lightning	基于磁盘的分布式行存 (Spanner)	Paxos+2PC	Log-Structured Merge tree	基于磁盘的分布式列存
TiDB	基于磁盘的分布式行存 (TiKV)	Raft+2PC	B-tree + Columnar delta tree	基于磁盘的分布式列存 (TiFlash)

(c) 磁盘型与分布式列存



(c) 磁盘型行存与分布式列存

优点:

负载隔离性**高**

OLAP吞吐量**高**

OLAP扩展性**高**

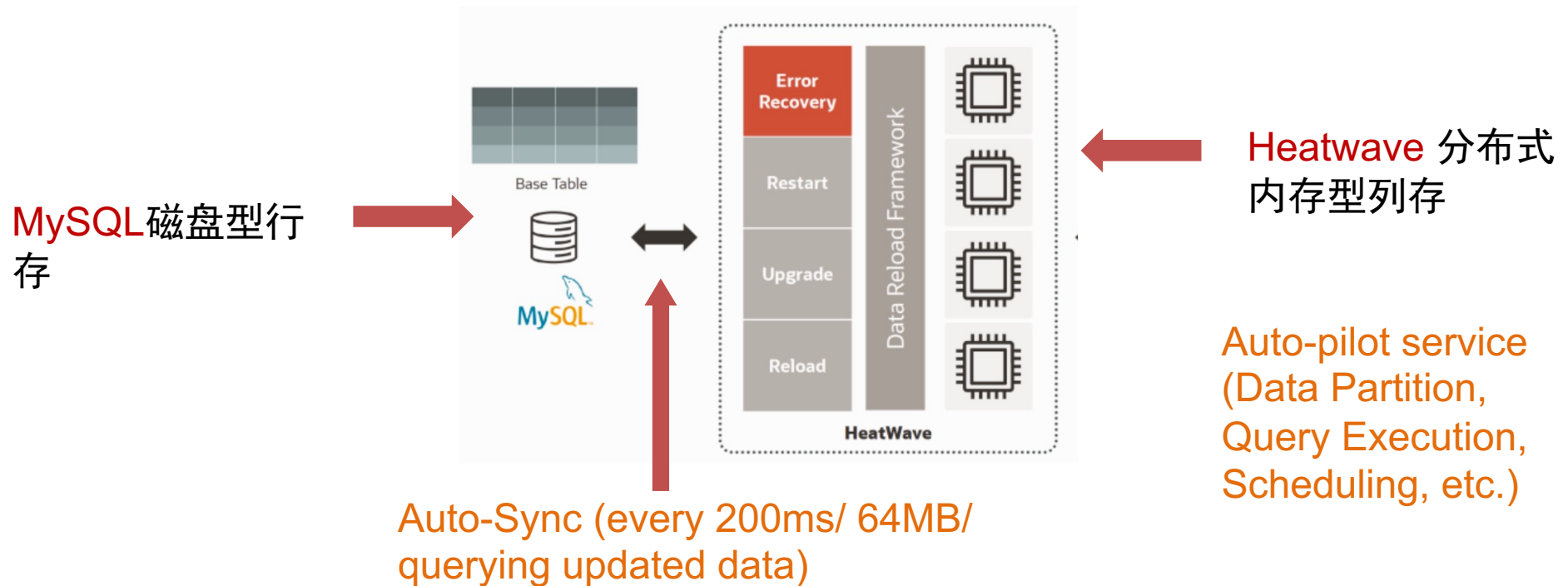
缺点:

数据新鲜度**低**

场景: 高AP吞吐量与低扩展性应用

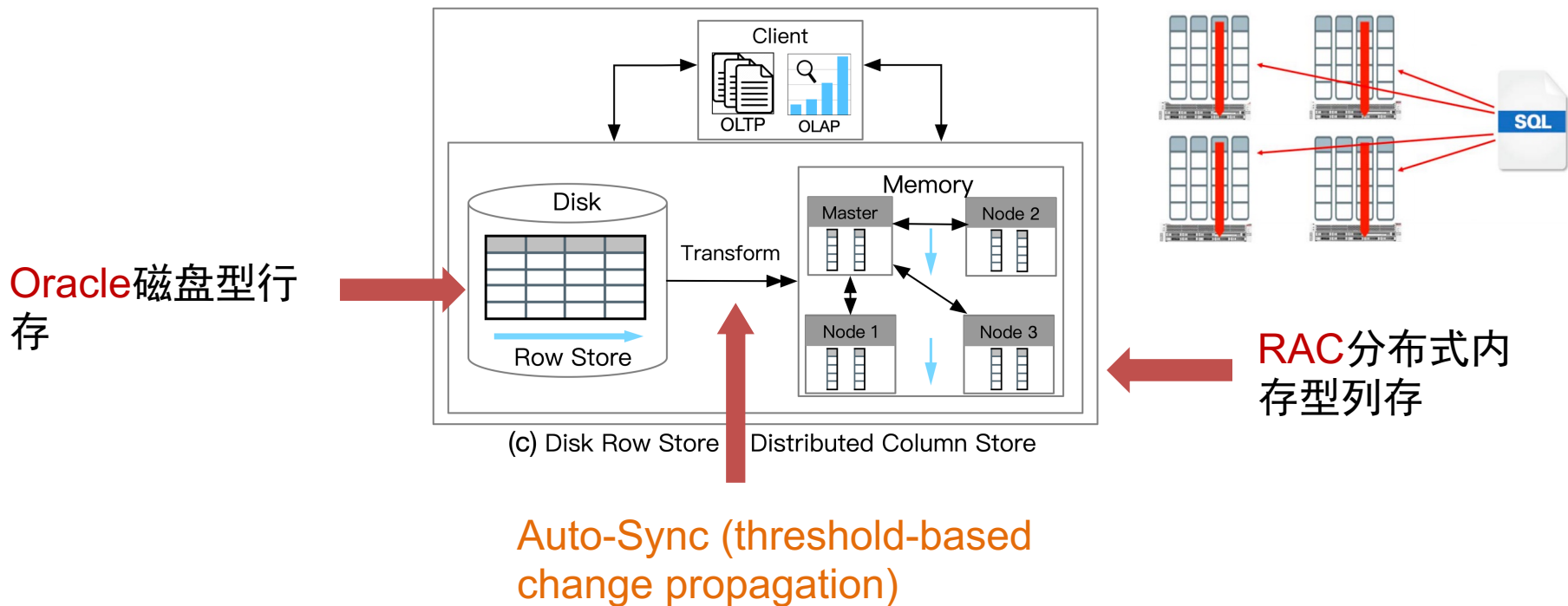
如**实时分析型物联网应用**

(c)类架构系统: MySQL Heatwave



MySQL Heatwave. Real-time Analytics for MySQL Database Service, August 2021, Version 3.0

(c)类架构系统: Oracle RAC



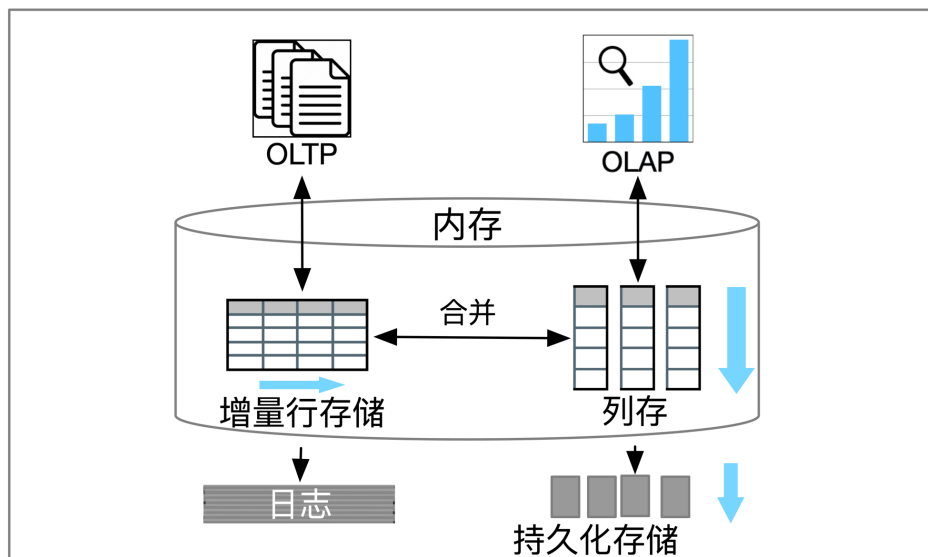
Lahiri, Tirthankar, et al. "Oracle database in-memory: A dual format in-memory database." In ICDE, 2015.



基于(c)类架构的HTAP数据库比较

HTAP 数据库	基于行存的OLTP	增量存储(Delta)	基于列存的OLAP	持久化列存	自动化管理
MySQL Heatwave	磁盘型行存	缓存	分布式内存型列存(Heatwave)	是	是 (Data Partition, Query Execution, Scheduling, etc.)
Oracle RAC	磁盘型行存	Snapshot Metadata Unit (SMU)	分布式内存型列存(RAC)	否	是 (Data Load and Eviction, Query Execution, etc.)

(d) 主列存与增量型行存



(d) 主列存与增量型行存

优点:

数据新鲜度**高**

OLAP吞吐量**高**

缺点:

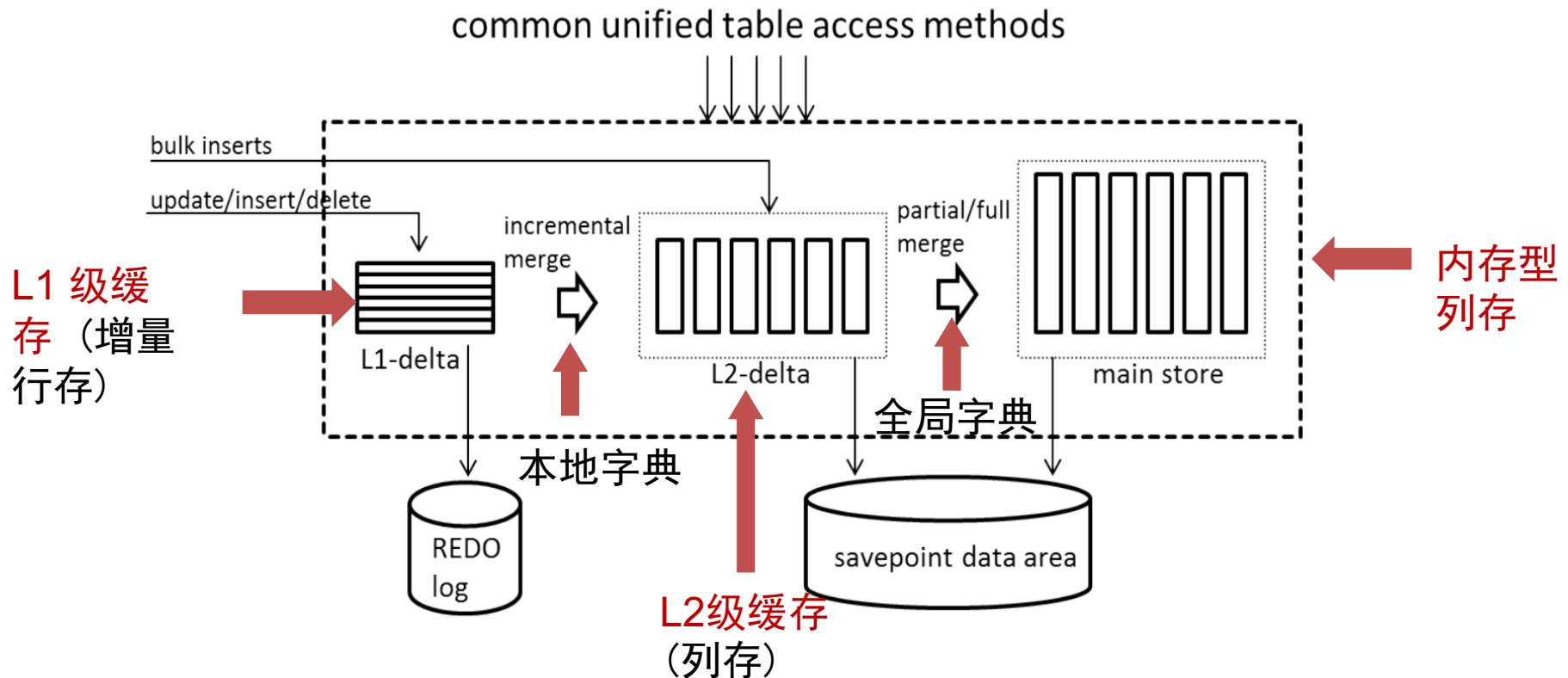
OLTP扩展性**低**

负载隔离性**低**

场景: 高AP吞吐量与高数据新鲜度

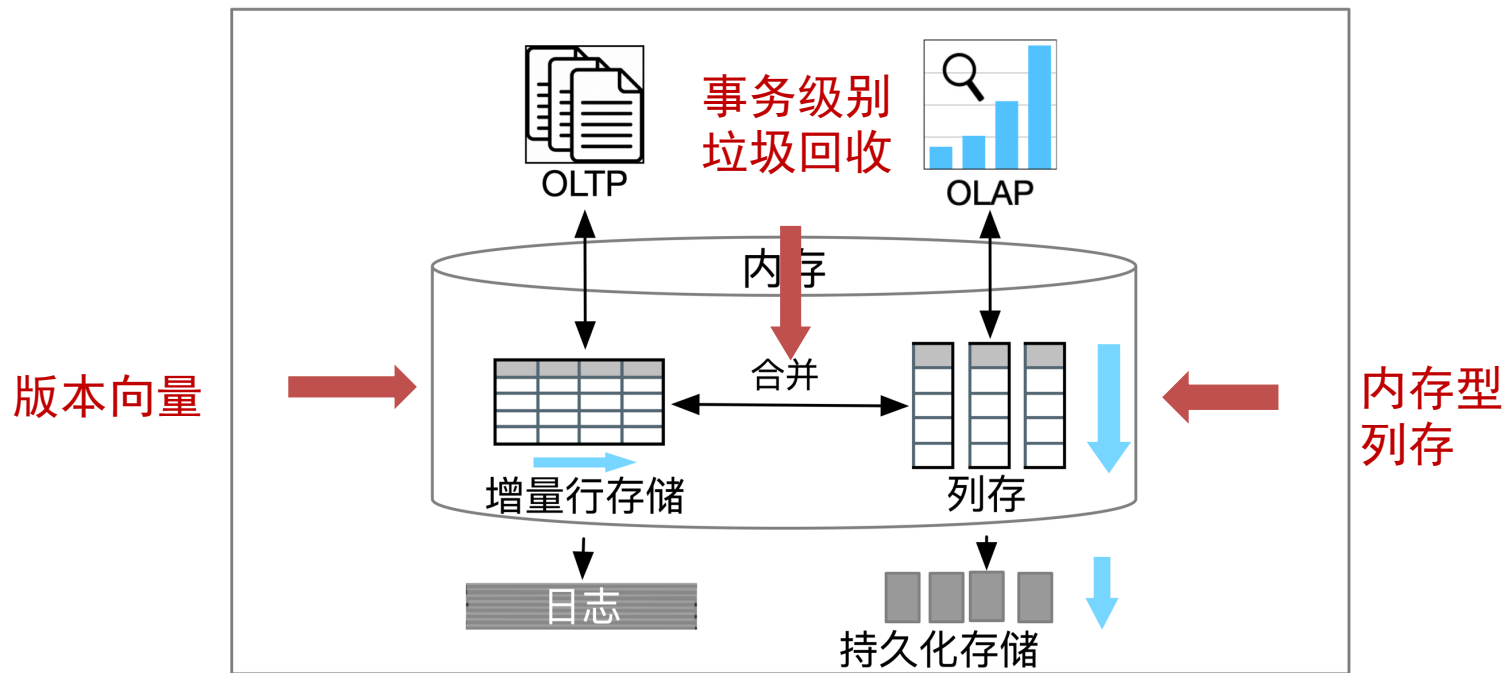
如**实时欺诈与异常检测应用**

(d)类架构系统: SAP HANA



Sikka, Vishal, et al. "Efficient transaction processing in SAP HANA database: the end of a column store myth." In *SIGMOD*. 2012.

(d)类架构系统: Hyper (Column)



(d) 主列存与增量型行存

Neumann, Thomas, Tobias Mühlbauer, and Alfons Kemper. "Fast serializable multi-version concurrency control for main-memory database systems." *In SIGMOD*, 2015.



基于(d)类架构的HTAP数据库比较

HTAP 数据库	基于行存的 OLTP	增量存储 (Delta)	基于列存的 OLAP	数据同步
SAP HANA	L1 Cache	L2 Cache	内存型列存	基于字典的同步
Hyper	Buffer	版本向量	内存型列存	事务级别 垃圾回收

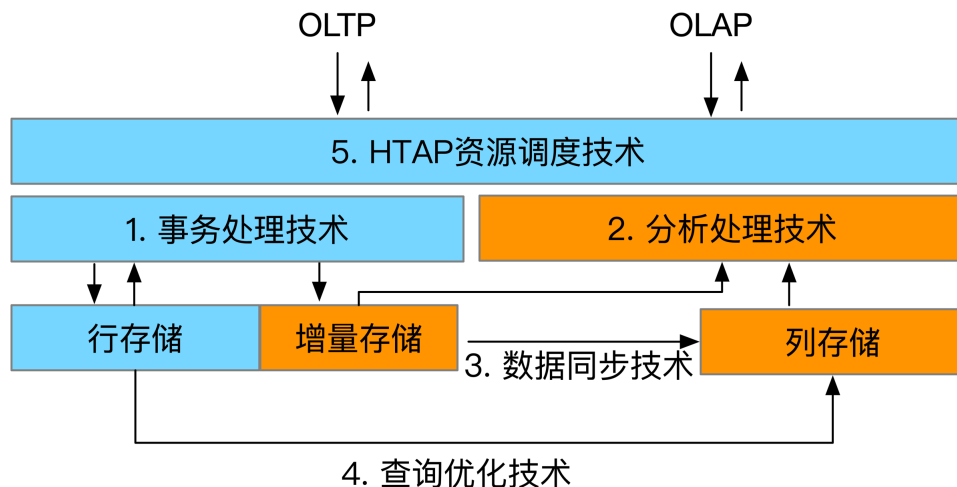


HTAP数据库架构的总结

架构分类	HTAP 数据库	OLTP 吞吐量	OLAP 吞吐量	OLTP 扩展性	OLAP 扩展性	负载隔 离性	数据新 鲜度
主行存与内存型列存	Oracle, SQL Server	高	中	高	低	低	高
分布式行存与列存副 本	TiDB, F1 Lightning	中	高	中	高	高	低
磁盘型行存与 分布式列存	MySQL Heatwave Oracle RAC	中	中	高	高	高	中
主列存与 增量型行存	SAP HANA Hyper	中	低	高	中	低	高

HTAP数据库关键技术总览

1. 事务处理技术：如何高效地更新行存储与增量存储
2. 分析处理技术：如何高效地扫描列存储与增量存储
3. 数据同步技术：如何高效地扫描合并增量存储的数据到列存储
4. 查询处理技术：如何合理地为查询算子选择数据访问路径（行存或列存）
5. 资源调度技术：如何合理地为混合负载分配资源（CPU或内存）





HTAP事务处理技术

1. 基于单机型行存与内存型增量存储的事务处理

- ❑ 基于MVCC的单机事务处理
- ❑ 基于内存型增量存储的数据更新
- ❑ 例如Oracle, SQL Server, SAP HANA

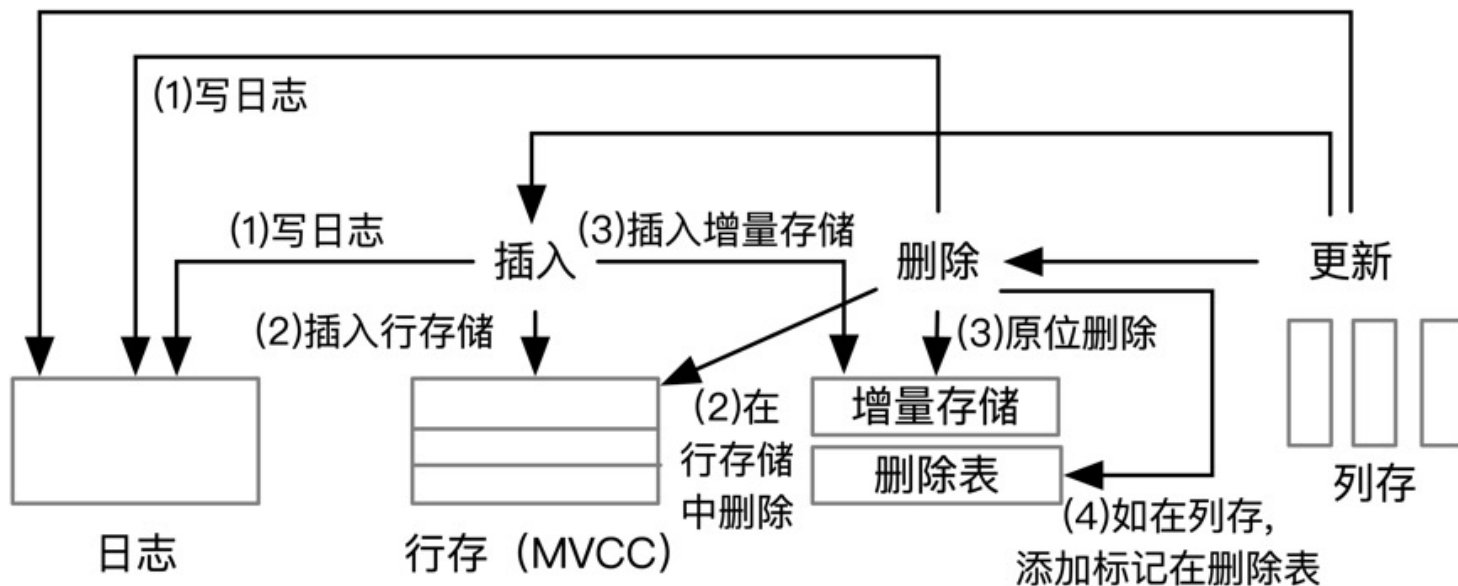
2. 基于分布式行存与日志回放的事务处理

- ❑ 基于2PC+Paxos的分布式事务处理与数据复制机制
- ❑ 基于日志回放的行/列存储数据更新
- ❑ 例如, F1 Lightning, TiDB

基于主从架构的复制型事务处理, 例如Singlestore

HTAP事务处理技术

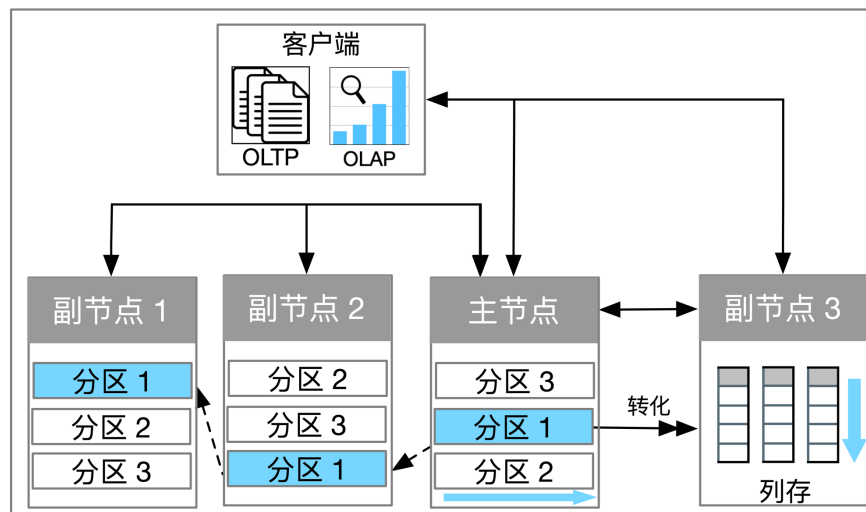
1. 基于单机型行存与内存型增量存储的事务处理



HTAP事务处理技术

2.1 基于主从复制的分布式事务处理

- ❑ 主节点首先处理事务, 然后复制数据到从节点
- ❑ 例如SingleStore采取基于内存的主从复制型分布式事务处理, 数据写到磁盘时候转换为列存



(b) 分布式行存与列存副本



HTAP数据库事务处理技术对比

技术类型	数据库	事务处理技术	增量存储	主要优点	主要缺点
基于单机型行存与内存型增量存储的事务处理	Oracle, SQL Server	MVCC	增量内存表	性能高	扩展性低
基于分布式行存与日志回放的事务处理	SingleStore	单机事务+主从同步复制	增量日志	性能高	新鲜度低
	TiDB, F1 Lightning	2PC+Paxos	增量	扩展性高	延迟高



HTAP分析处理技术

1. 基于内存型增量存储与单机型列存储的扫描

- ❑ 单指令-多数据流(SIMD), 向量执行
- ❑ 基于内存的增量存储扫描
- ❑ 例如Oracle, SQL Server

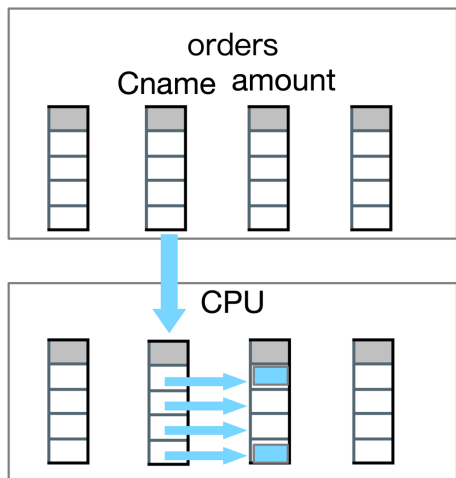
2. 基于增量日志与分布式列存储的扫描

- ❑ 基于列式文件的分布式查询
- ❑ 多阶段增量日志文件合并
- ❑ E.g., F1 Lightning, TiDB

HTAP分析处理技术

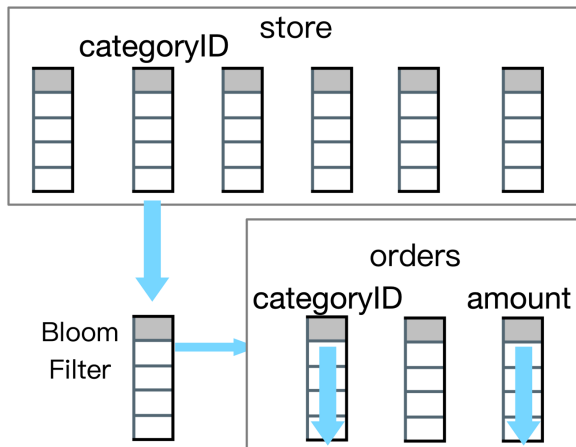
1. 基于内存型增量存储与单机型列存储的扫描

Q1: SELECT amount From
orders Where Cname='JASON'



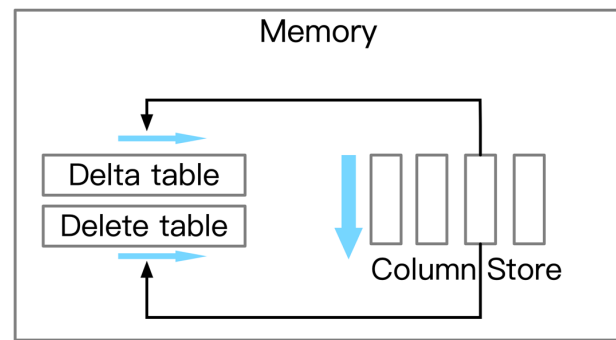
(a) SIMD query processing

Q2: Select SUM(amount)
From store s, orders o
Where s.categoryID=o.categoryID



(b) Vector join based on a bloom filter

获取增量存储中的可见
数据并且跳过删除表中
的数据

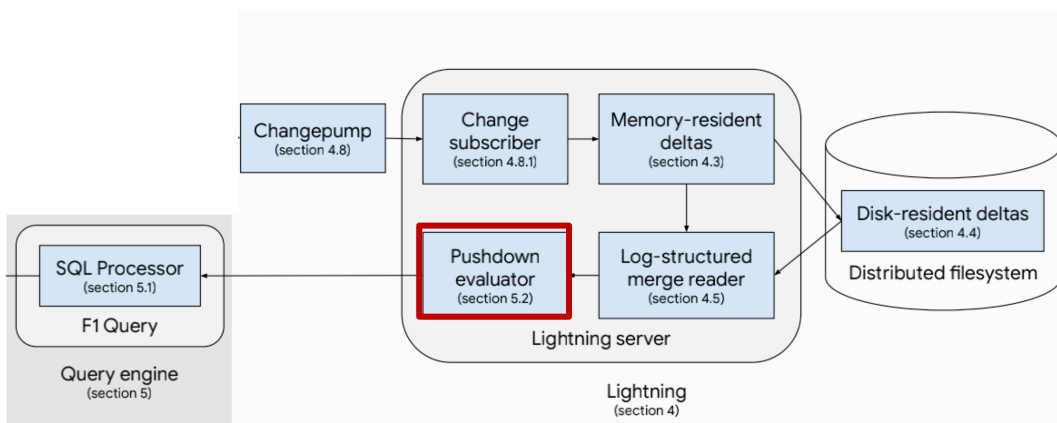


(c) Column Scan with delta traversing

HTAP分析处理技术

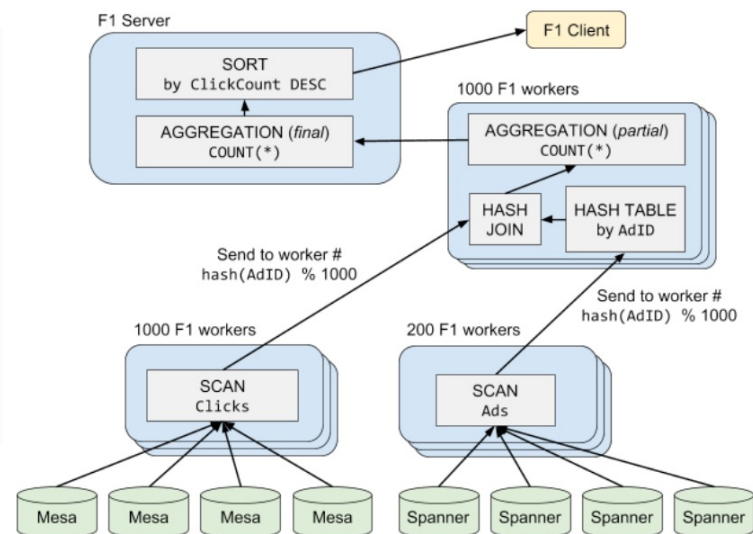
2. 基于增量日志与分布式列存储的扫描

步骤一：增量日志文件合并与扫描



Samwel, Bart, et al. "F1 query: Declarative querying at scale." *PVLDB* 11(12), 2018: 1835-1848.

步骤二：分布式列式扫描



Yang, Jiacheng, et al. "F1 Lightning: HTAP as a Service." *PVLDB* 13(12), 2020: 3313-3325.



HTAP数据库分析处理技术对比

技术类型	数据库	分析处理技术	增量存储	主要优点	主要缺点
基于内存型增量存储与单机型列存储的扫描	Oracle, SQL Server, SAP HANA	SIMD+ 增量存储表扫描	增量内存表	数据新鲜度高	内存空间大
基于增量日志与分布式列存储的扫描	TiDB, F1 Lightning	分布式列式扫描 + 多阶段日志合并 与扫描	增量内存表 与日志文件	扩展性高	延迟高



HTAP数据同步技术

1. 基于内存增量表与内存型列存的数据同步

- (1) 基于阈值驱动的数据同步技术
- (2) 两阶段迁移的数据同步技术
- (3) 字典排序合并的数据同步技术

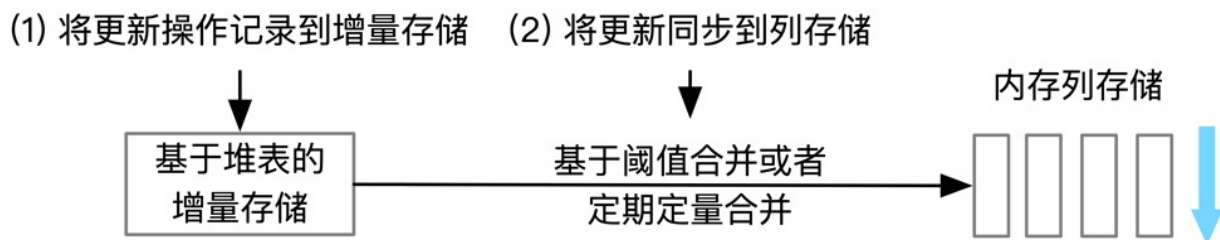
2. 基于增量日志与持久化列存的数据同步

- LSM-tree and B-tree
- TiDB, F1 Lightning

HTAP数据同步技术

1. 基于内存增量表与内存型列存的数据同步

- ❑ 方法一：基于阈值驱动的数据同步技术
- ❑ 例如，增量数据达到列存数据的90%，或者按照200ms间隔



基于阈值驱动的数据同步技术

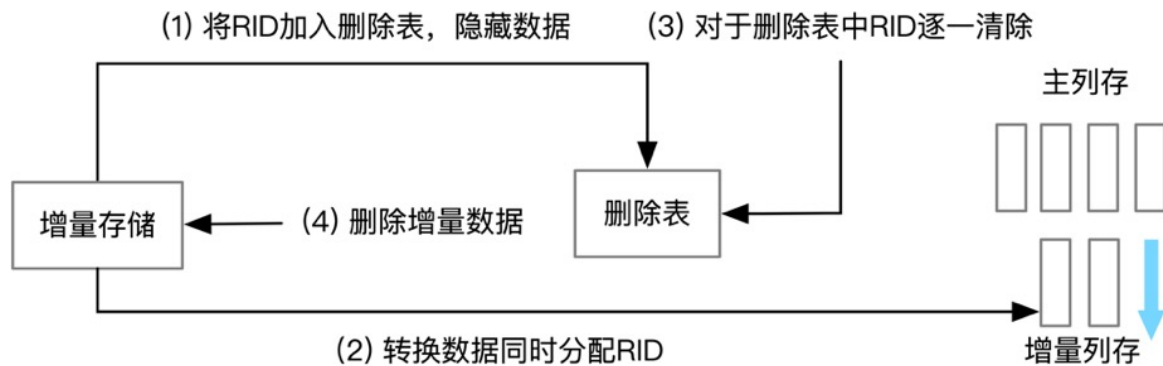
HTAP数据同步技术

1. 基于内存增量表与内存型列存的数据同步

❑ 方法二：两阶段迁移的数据同步技术

❑ 阶段一：迁移准备，步骤(1)与(2)，数据暂时不可见

❑ 阶段二：迁移执行，步骤(3)与(4)，增量数据被迁移至列存

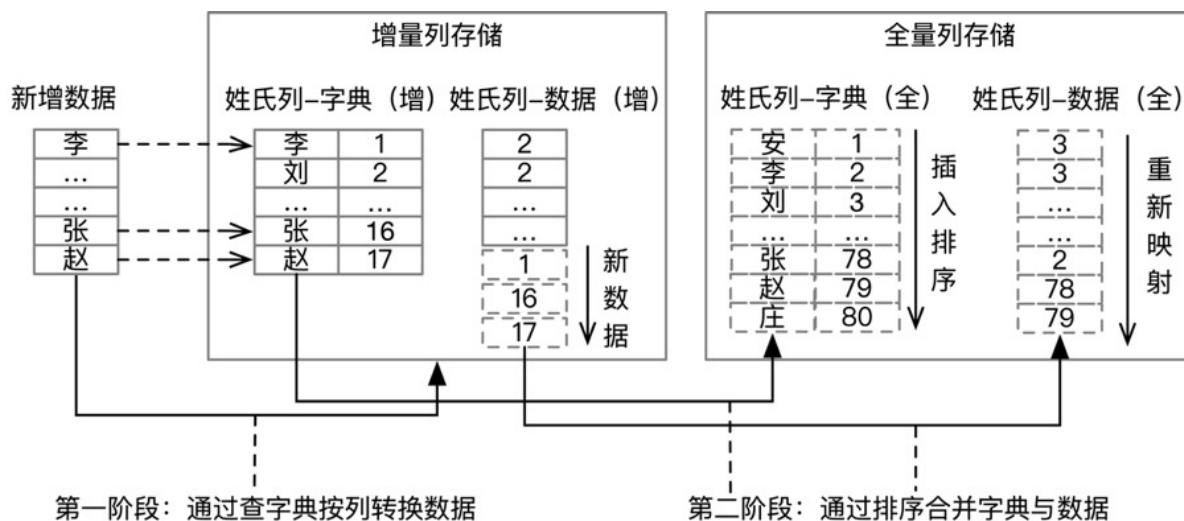


两阶段迁移的数据同步技术

HTAP数据同步技术

1. 基于内存增量表与内存型列存的数据同步

- ❑ 方法三：两阶段迁移的数据同步技术
- ❑ 基于本地字典与全局字典的数据同步

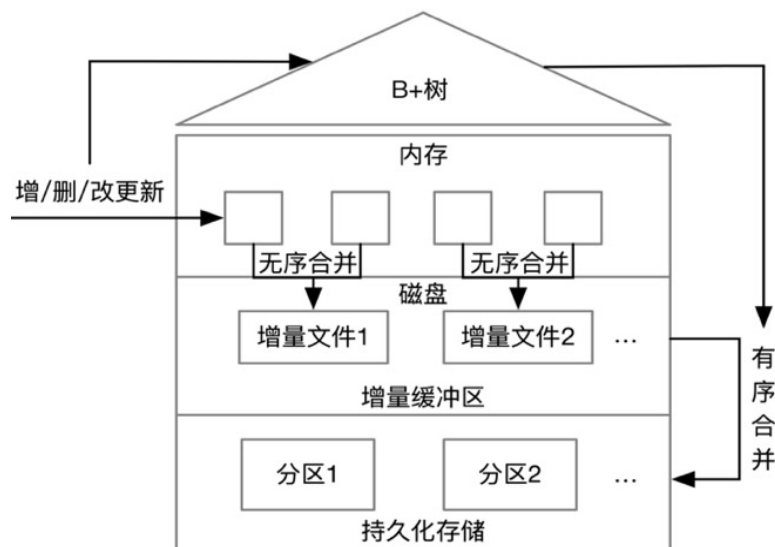


字典排序合并的数据同步技术

HTAP数据同步技术

2. 基于增量日志与持久化列存的数据同步

- ❑ 基于内存的面向行的增量存储
- ❑ 基于磁盘的面向列的增量存储
- ❑ 多阶段合并：内存deltas无序合并至增量文件，然后有序合并至持久化存储





HTAP数据库数据同步技术对比

技术类型	数据库	数据同步技术	主要优点	主要缺点
基于内存增量表与内存型列存的数据同步	Oracle, SQL Server, SAP HANA	基于阈值驱动的数据同步技术、两阶段迁移的数据同步技术、字典排序合并的数据同步技术、	性能高	扩展性低
基于增量日志与持久化列存的数据同步	TiDB, F1 Lightning	多阶段deltas与日志文件合并, 基于B+tree的有序合并	扩展性低	合并代价高



HTAP查询优化技术

1. 基于主行存的内存列选择

- 以存储层的内存列加速的方式针对频繁使用的查询进行加速

2. 混合行/列存储扫描

- 查询执行层面针对查询的物理计划进行优化

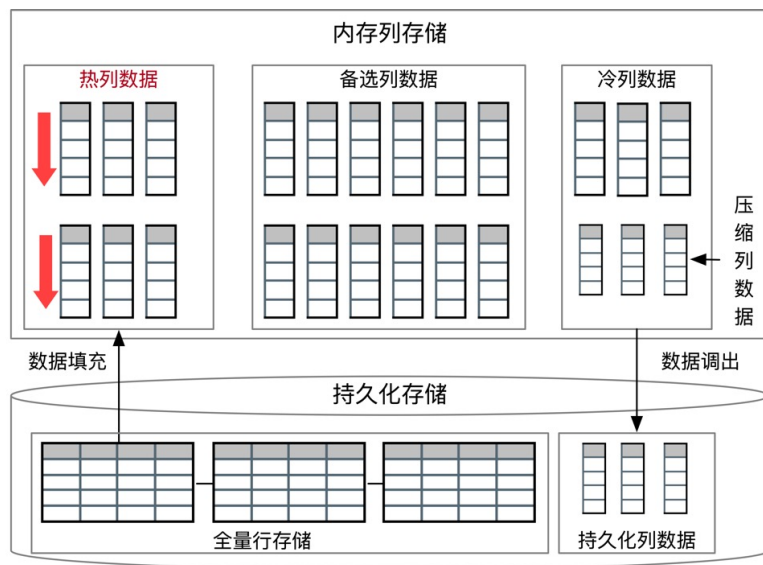
3. 异构CPU/GPU硬件加速

- 在处理器层面以CPU/GPU的异构处理器技术加速HTAP的处理

HTAP查询优化技术

1. 基于主行存的内存列选择

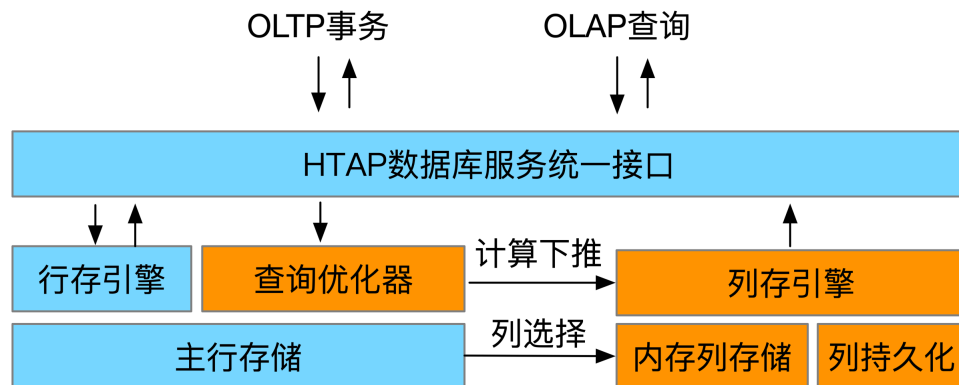
- 主要挑战在于如何选择更重要的列数据到有限的内存里
- 主要方法为基于热力图的方法（热列数据，备选列数据，以及冷列数据）
- 基于列的访问频率选择列



HTAP查询优化技术

2. 混合行/列存储扫描技术

- 对于一个查询的算子集合，一部分在行存储引擎里执行，另一部分在列存储引擎里执行
- 方法一：基于规则的方法
- 方法二：基于代价的方法

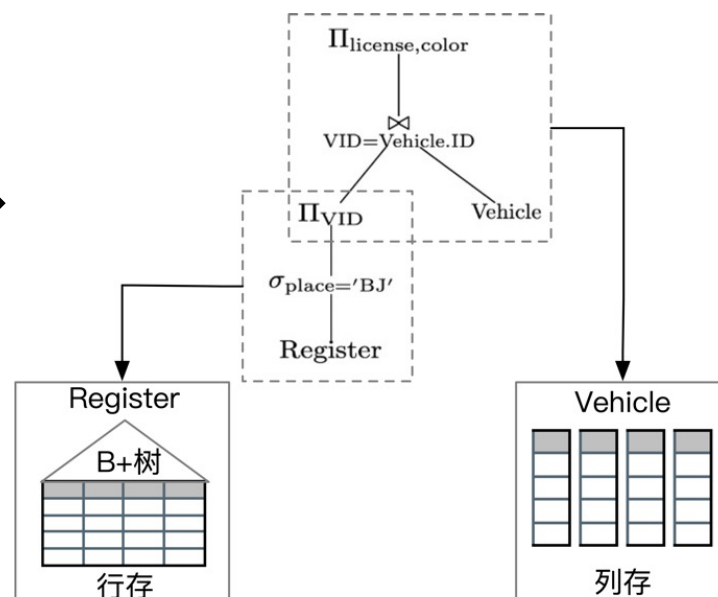
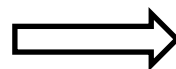


HTAP查询优化技术

2. 混合行/列存储扫描技术

- 基于规则的方法：先扫描列存，然后扫描行存，如Oracle, SQL server

SELECT license, color
FROM Register R, Vehicle V
WHERE R.VID=V.ID and R.place='BJ'



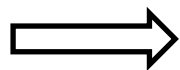


HTAP查询优化技术

2. 混合行/列存储扫描技术

- 基于代价函数的方法, e.g., TiDB
- 比较算子在行存/索引与列存中的扫描代价

SELECT T.*, S.a
FROM T, S
WHERE T.b=S.b
and 1<T.a<100



$$C_{optimal} = \min(C_{column}, C_{row}, C_{index}) \quad (1)$$

$$C_{column} = \sum_{i=1}^n (\bar{S}_c \cdot \check{S}_i \cdot C_{scan_column} + \check{S}_i \cdot C_{seek_column}) \quad (2)$$

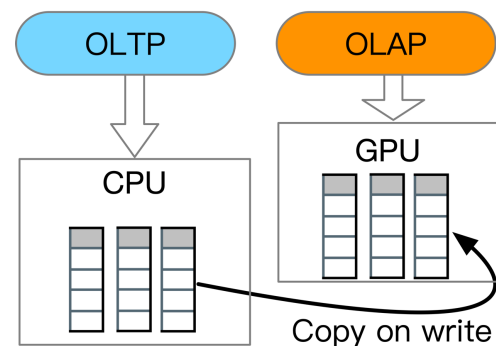
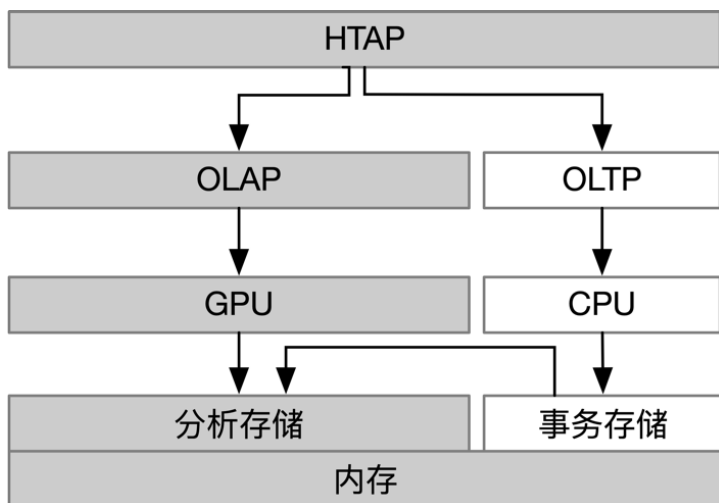
$$C_{row} = \bar{S}_r \cdot \check{S}_r \cdot C_{scan_row} + \check{S}_r \cdot C_{seek_row} \quad (3)$$

$$C_{index} = \bar{S} \cdot \check{S} \cdot C_{scan_index} + \check{S} \cdot C_{seek_index} + C_{double_read} \quad (4)$$

HTAP查询优化技术

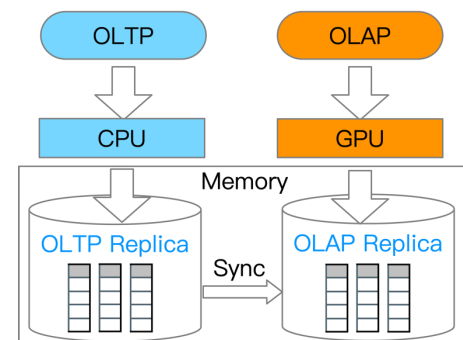
3.异构CPU/GPU硬件加速

- CPU任务并行的特点处理OLTP事务
- 利用GPU数据并行的特点处理OLAP分析



Shared Memory

写复制技术 (Caldera)



基于主备存储的隔离执行 (RateupDB)



HTAP数据库查询优化技术对比

技术类型	数据库	关键技术	主要优点	主要缺点
基于主行存的内存列选择	Oracle	热力图	性能高	性能抖动
混合行/列存储扫描	Oracle	基于规则的计划选择	性能高	本地搜索
	TiDB	基于代价函数的计划选择	全局搜索	独立性假设
异构CPU/GPU硬件加速	Caldera	写复制技术	新鲜度高	吞吐量低
	RateupDB	基于主备存储的隔离执行	隔离性高	新鲜度低



HTAP资源调度技术

1. 基于负载驱动的资源调度

- 基于HTAP负载的执行状态调度资源
- SAP HANA

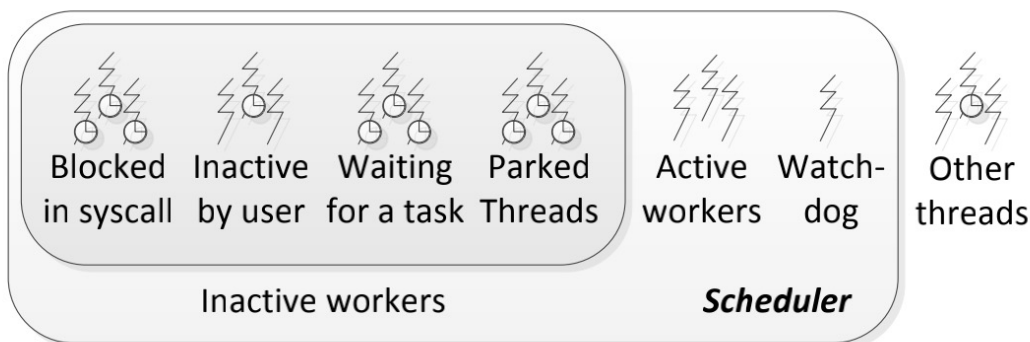
2. 基于新鲜度驱动的资源调度

- 切换混合负载执行的状态
- 需要在负载隔离性与数据新鲜度之间寻找一个平衡点

HTAP资源调度技术

1. 基于负载驱动的资源调度

□ 根据watchdog观测，分配更多资源给OLTP 或者OLAP



Psaroudakis, Iraklis, et al. "Task scheduling for highly concurrent analytical and transactional main-memory workloads." *In ADMS*, 2013.



HTAP资源调度技术对比

技术类型	代表性方法	关键技术	主要优点	主要缺点
基于负载驱动的资源调度	SAP HANA	基于规则的资源调度	性能高	新鲜度高
基于新鲜度驱动的资源调度	RDE [Raza, Aunn, et al SIGMOD 20]	基于阈值的执行模式切换	新鲜度高	性能不高



HTAP评测基准

□ CH-Benchmark

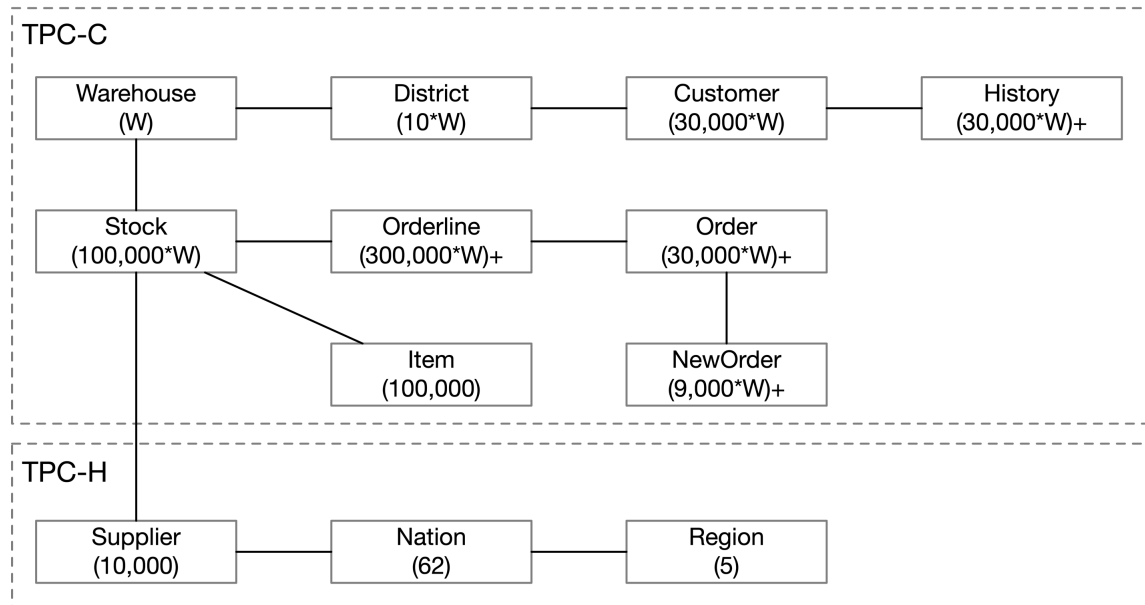
- TPC-C + TPC-H
- 可控的混合OLTP/OLAP执行模式
- 统一评测指标

□ HTAPBench

- TPC-C + TPC-H
- 固定OLTP指标结合可控的OLAP执行
- 时间窗口Time Window
- 统一评测指标

CH-Benchmark

- 基于TPC-C与TPC-H的统一数据模式
- 12个表（结合TPC-C的9个表与TPC-H的8个表）
- TPC-H使用与TPC-C一样的扩展因子用于数据生成





CH-Benchmark

□ TPC-C 的5个事务

- New Order, Payment, Order-Status, Delivery, and Stock-Level.

```
SELECT n_name, SUM(ol_amount) AS revenue
FROM customer, "order", orderline, stock, supplier,
     nation, region
WHERE c_id=o_c_id AND c_w_id=o_w_id AND c_d_id=o_d_id
     AND ol_o_id=o_id AND ol_w_id=o_w_id
     AND ol_d_id=o_d_id
     AND ol_w_id=s_w_id AND ol_i_id=s_i_id
     AND mod((s_w_id * s_i_id),10000)=su_suppkey
     AND ascii(SUBSTRING(c_state, 1, 1))=su_nationkey
     AND su_nationkey=n_nationkey
     AND n_regionkey=r_regionkey
     AND r_name='[REGION]' AND o_entry_d>='[DATE]'
GROUP BY n_name ORDER BY revenue DESC
```

□ 修改的TPC-H的22查询

- 修改了相应的表名与连接键,
- 简化了算术运算

Listing 1: CH-benCHmark query 5

```
SELECT n_name, SUM(l_extendedprice * (1 - l_discount)
) AS revenue
FROM customer, orders, lineitem, supplier, nation,
     region
WHERE c_custkey = o_custkey
     AND l_orderkey = o_orderkey
     AND l_suppkey = s_suppkey
     AND c_nationkey = s_nationkey
     AND s_nationkey = n_nationkey
     AND n_regionkey = r_regionkey
     AND r_name = '[REGION]'
     AND o_orderdate >= DATE '[DATE]'
     AND o_orderdate < DATE '[DATE]' + INTERVAL '1' YEAR
GROUP BY n_name ORDER BY revenue DESC
```

□ 移除了TPC-H的刷新函数

Listing 2: TPC-H Query 5



CH-Benchmark

□ 执行规则

- 单模式执行OLTP/OLAP 或者混合执行 (OLTP+OLAP)
- 主要参数为OLTP的OLAP并发流数目

□ Metrics评测指标

- OLAP参照系指标

$$M(OLAP) = \frac{\text{tpmC}}{Q_{\text{phH}}} @ Q_{\text{phH}}$$

- OLTP参照系指标

$$M(OLTP) = \frac{\text{tpmC}}{Q_{\text{phH}}} @ \text{tpmC}$$

- 当OLTP为主要参照对象，顺序执行混合负载的指标为**5.7@5084 tpmC**，然后并行执行混合负载的指标为**6.5@5188 tpmC**，那么可以认为系统在并行执行混合负载后性能没有下降，且有略微的提升

HTAPBench

□ 数据模式与工作负载

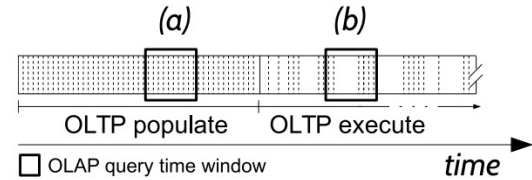
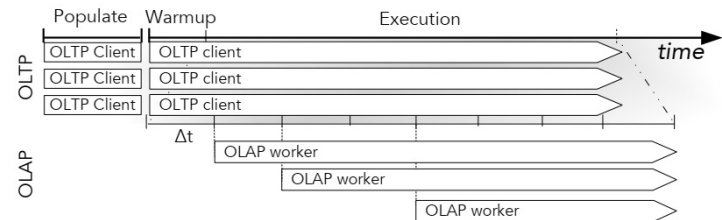
- 同样基于TPC-C与TPC-H

□ 执行规则

- 固定tpmC 结合OLAP工作数
- 为查询计算动态时间窗口访问最新插入数据

□ 评测指标

- 在固定OLTP指标下, 每个OLAP工作每小时所能响应的查询数目



$$QpHpW = \frac{QphH}{\#OLAPworker} @tpmC$$



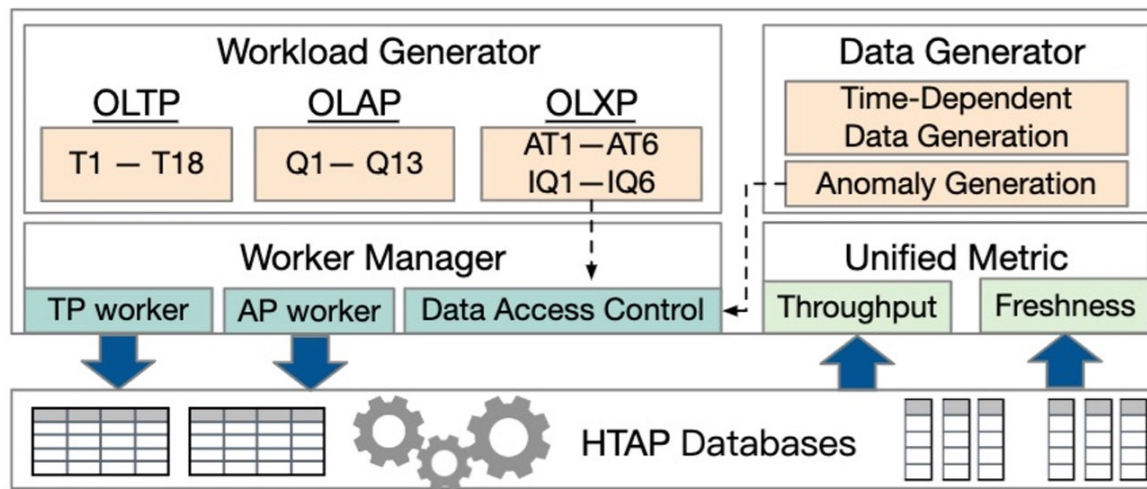
HTAP评测基准对比

HTAP 评测基准	数据生成与工作负载	关键技术	主要优点	主要缺点
CH-Benchmark	TPC-C+TPC-H	可控制混合执行, 参照系统一评测指标	灵活性高	数据新鲜度低
HTAPBench	TPC-C+TPC-H	固定OLTP的可控 OLAP执行, 动态时间窗口	数据新鲜度高	固定的OLTP指标

HTAP数据库基准

HyBench Overview

- **Data Generator**: time-dependent and anomaly generation
- **Workload Generator**: a hybrid workload of OLTP, OLAP, and OLXP with choke-point design
- **Unified Metric**: An E2E Evaluation of Freshness, TPS, QPS, XPS



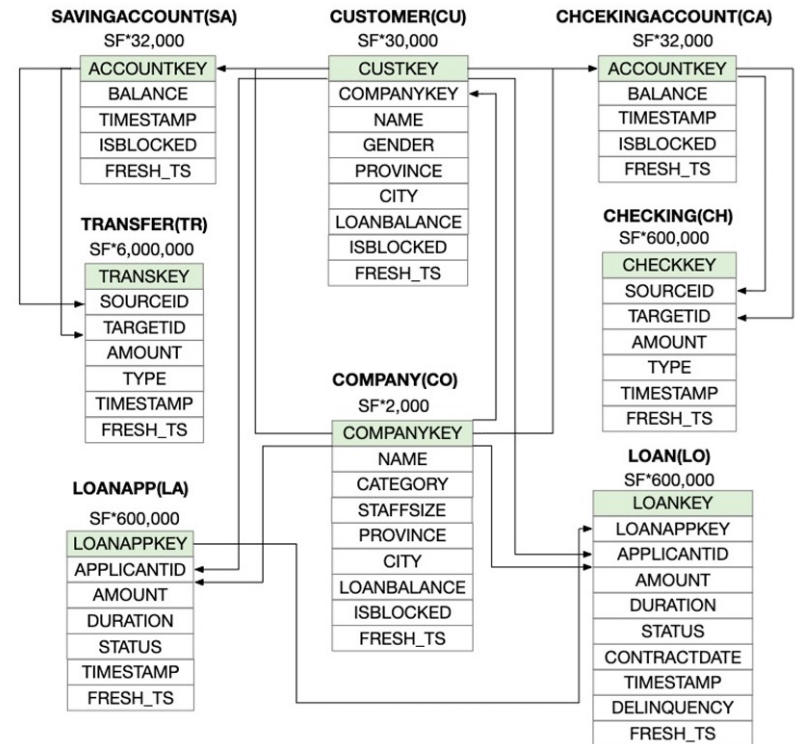
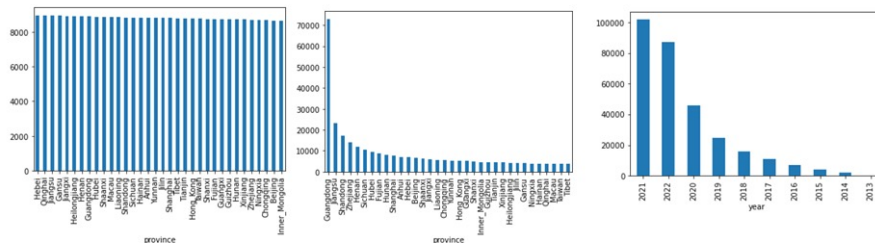
HTAP数据库基准

❑ Applications: On-line Finance APPs



Multi-Level: Customer->Company->Vendor

□ Distributions: Uniform and Skewed



HTAP数据库基准

□ The Case of Update:

–TP:{ (i1,ts1), (i2,ts2)}

–AP:{ (i1,ts3), (i2,ts4)}

□ The Case of Insert:

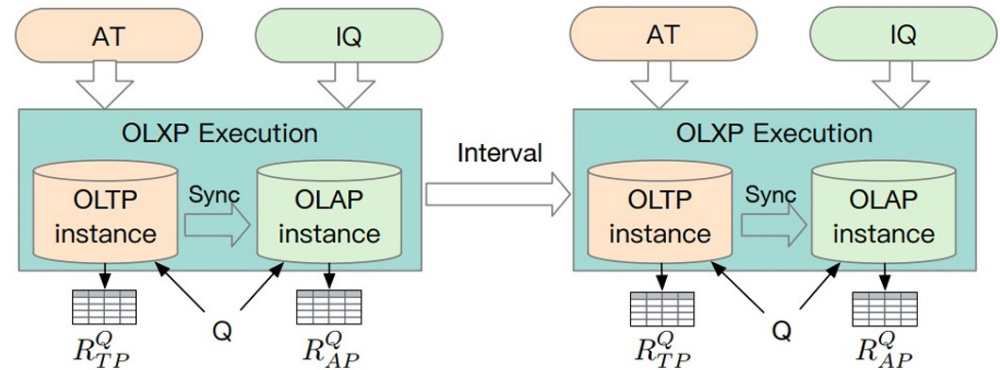
–TP:{ (i1,ts1), (i2,ts2), (i3,ts3)}

–AP:{ (i1,ts4), (i2,ts5)}

□ The Case of Delete:

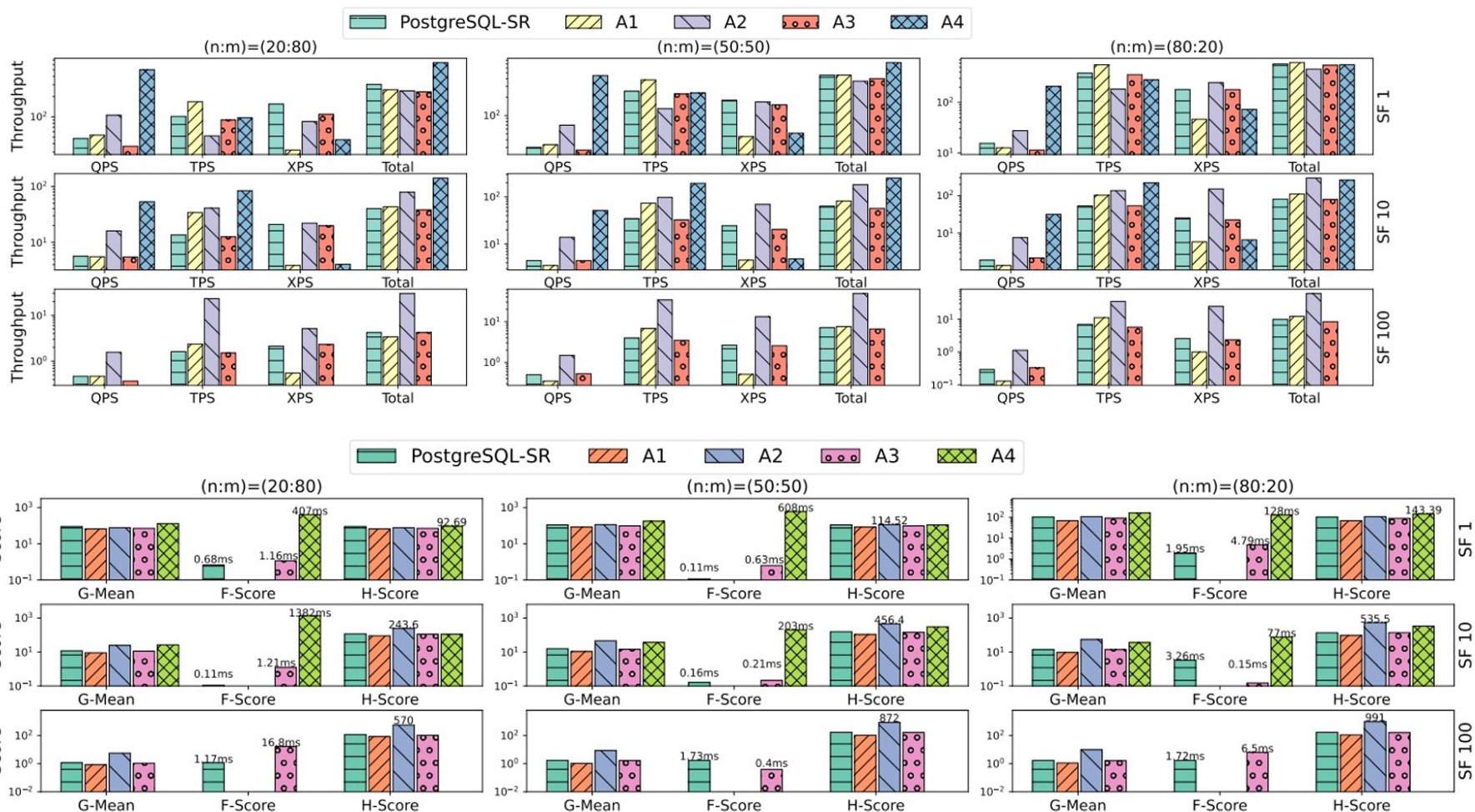
–TP:{ (i1,ts4), (i2,ts5)}

–AP:{ (i1,ts1), (i2,ts2), (i3,ts3)}



$$\overline{f_s} = \max \begin{cases} ts_{r_a} - ts_{r_b} | r_a \in R_{TP}^Q, r_b \in R_{AP}^Q, i_{r_a} = i_{r_b} & \text{update} \\ ts_Q - ts_{r_a} | r_a \in R_{TP}^Q, r_b \notin R_{AP}^Q, i_{r_a} = i_{r_b} & \text{insert} \\ ts_Q - ts_{r_a} | r_a \notin R_{TP}^Q, r_b \in R_{AP}^Q, i_{r_a} = i_{r_b} & \text{delete} \end{cases}$$

HTAP数据库基准





HTAP数据库总结

- **四种存储架构**: (1)主行存与内存型列存;(2)分布式行存与列存副本;(3)磁盘型行存与分布式列存;(4)以及主列存与增量型行存
- **五种关键技术**: (1)事务处理技术;(2)分析处理技术;(3)数据同步技术;(4)查询优化技术;(5)资源调度技术;
- **三个常用评测基准**: (1) CH-Benchmark; (2) HTAPBench; (3) HyBench
- **参考文献**:
 - **Chao Zhang**, Guoliang Li, et al. “HTAP Databases: A Survey.” IEEE Transactions on Knowledge and Data Engineering (2024).
 - **Chao Zhang**, Guoliang Li, and Tao Lv. “HyBench: A New Benchmark for HTAP Databases.” Proceedings of the VLDB Endowment 17.5 (2024): 939-951
 - Haowen Dong, **Chao Zhang**, Guoliang Li, et al. “Cloud-Native Databases: A Survey.” IEEE Transactions on Knowledge and Data Engineering (2024).



谢谢！



中國人民大學
RENMIN UNIVERSITY OF CHINA